

2025년 상반기 K-디지털 트레이닝

예외 처리

[KB] IT's Your Life

1 예외와 예외 클래스

✓ 예외와 에러

반드시 예외처리(try/catch, throws)해주어야 하는 상황

-
- 자바에서 외부 자원을 연결해야할 때
 - File연결
 - db연결
 - 네트워크 연결
 - 컴퓨터 주요 device연결(cpu)
- > 예외처리를 왜 하는가?

프로그램이 중단되는 상황이 생기면 더이상 실행하지 않고,
프로그램이 멈추어버린다.

중단(방해, interrupt, 인터럽트)

1 예외와 예외 클래스

✓ 1) 예외처리를 왜 하는가? 안하면 어떻게 되는가?

==> 실행할 때 특정한 부분에서 에러가 나면 프로그램이 중단되어 버린다.
특정 부분의 실행을 포기하고 프로그램을 중단시키지 않기 위해서

✓ 2) 에러는 컴파일에러, 실행에러 비교!!

==> 컴파일(번역) 에러: 영어코드(문법에러)---번역---->JVM

==> 실행(run-time) 에러: 논리적 문제, 자원 연결 문제 JVM이 실행(****)

✓ 3) 자바에서 예외처리하는 방법:

```
try {  
    에러가 발생할 것 같은 코드  
} catch(Exception e) {  
    에러가 발생했을 때 처리 내용 코드  
}
```

✓ 4) 반드시 예외처리해야하는 경우는?

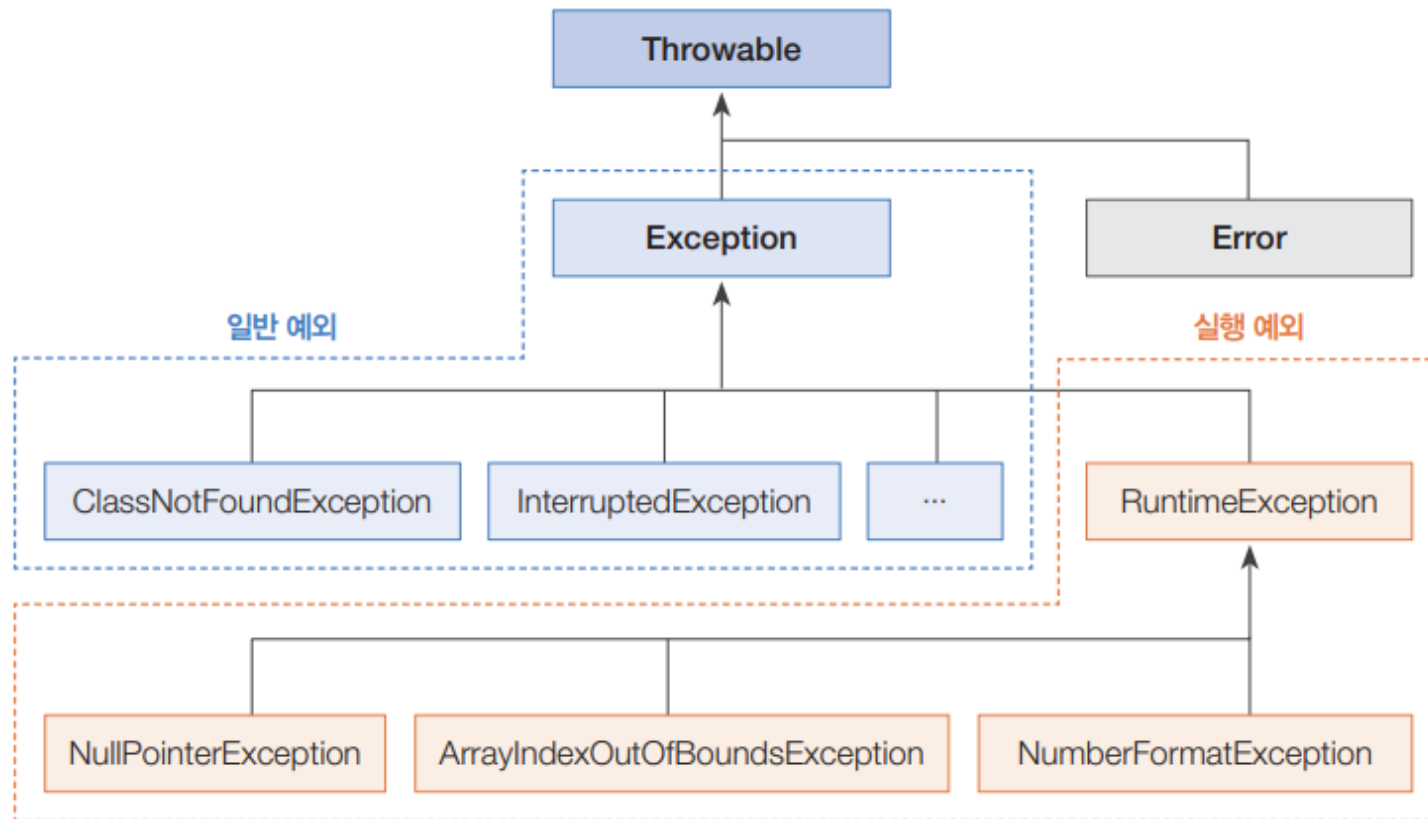
외부 자원을 연결할 때(스트림을 사용할 때): network, file, db, cpu

1 예외와 예외 클래스

✓ 예외와 에러

○ 예외: 잘못된 사용 또는 코딩으로 인한 오류

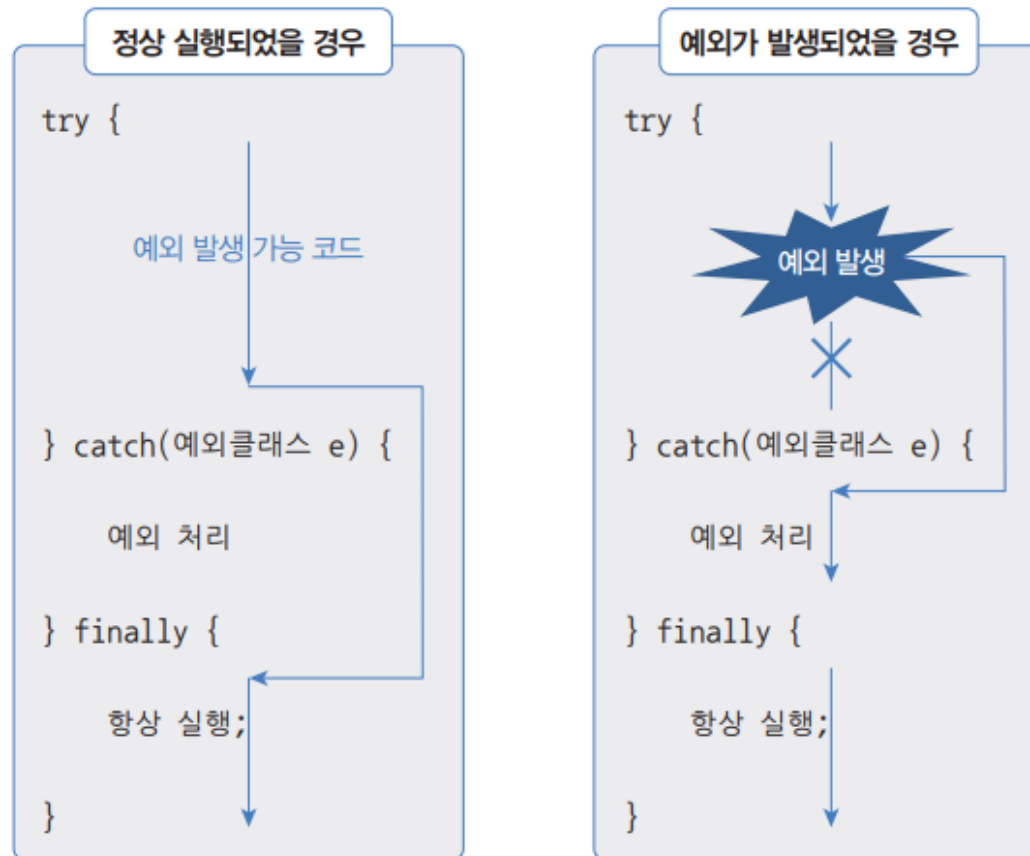
- 에러와 달리 예외 처리를 통해 계속 실행 상태를 유지할 수 있음
- 일반 예외(Exception): 컴파일러가 예외 처리 코드 여부를 검사하는 예외
- 실행 예외(Runtime Exception): 컴파일러가 예외 처리 코드 여부를 검사하지 않는 예외



2 예외 처리 코드

✓ 예외 처리

- 예외 발생 시 프로그램의 갑작스러운 종료를 막고 정상 실행을 유지할 수 있게 처리하는 코드
- 예외 처리 코드는 try-catch-finally 블록으로 구성
- try-catch-finally 블록은 생성자 내부와 메소드 내부에서 작성



2 예외 처리 코드

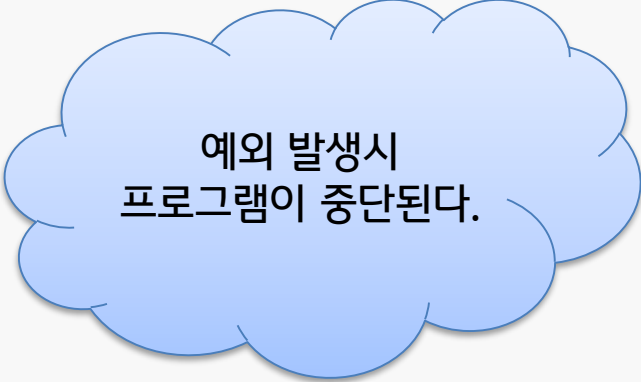
 ExceptionHandlingExample1.java

```
package ch11.sec02.exam01;

public class ExceptionHandlingExample1 {

    public static void printLength(String data) {
        int result = data.length(); // data가 null일 경우 NullPointerException 발생
        System.out.println("문자 수: " + result);
    }

    public static void main(String[] args) {
        System.out.println("[프로그램 시작]\n");
        printLength("ThisIsJava");
        printLength(null); // 매개값으로 null 대입
        System.out.println("[프로그램 종료]");
    }
}
```



예외 발생시
프로그램이 중단된다.

[프로그램 시작]

문자 수: 10

Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()" because "data" is null
at ch11.sec02.exam01.ExceptionHandlingExample1.printLength(ExceptionHandlingExample1.java:5)
at ch11.sec02.exam01.ExceptionHandlingExample1.main(ExceptionHandlingExample1.java:12)

2 예외 처리 코드

ExceptionTest.java

```
package 예외처리;

public class 문제발생클래스2 {

    public static void main(String[] args) {
        System.out.println("1. 나는 프린트가 잘 될 예정.");

        try {
            // 문제가 발생할 것은 코드를 넣어주는 부분.
            System.out.println("2. 문제발생 코드 " + 10/0);
        } catch (Exception e) {
            // 예외상황이 발생하면, 잡아서 어떻게 처리할 지 코드를 넣어주는 부분.
            System.out.println("에러 발생함.");
            System.out.println(e.getMessage());
        }

        System.out.println("3. 나는 과연 프린트가 될까요??");
    }
}
```

발생한 문제만 실행하
지 않는 예외라고 하고
이어서 실행할 수 있다.
try-catch이용

1. 나는 프린트가 잘 될 예정.
에러 발생함.
/ by zero
3. 나는 과연 프린트가 될까요??

2 예외 처리 코드

 ExceptionHandlingExample2.java

```

package ch11.sec02.exam01;

public class ExceptionHandlingExample2 {
    public static void printLength(String data) {
        try {
            int result = data.length();
            System.out.println("문자 수: " + result);
        } catch (NullPointerException e) {
            System.out.println(e.getMessage()); //①
            //System.out.println(e.toString()); //②
            //e.printStackTrace(); //③
        } finally {
            System.out.println("[마무리 실행]\n");
        }
    }

    public static void main(String[] args) {
        System.out.println("[프로그램 시작]\n");
        printLength("ThisIsJava");
        printLength(null);
        System.out.println("[프로그램 종료]");
    }
}

```



finally 블록은
에러 발생 여부와 상관
없이 무조건 실행

[프로그램 시작]

문자 수: 10
[마무리 실행]

Cannot invoke "String.length()" because "data" is null
[마무리 실행]

[프로그램 종료]

2 예외 처리 코드

 ExceptionHandlingExample.java

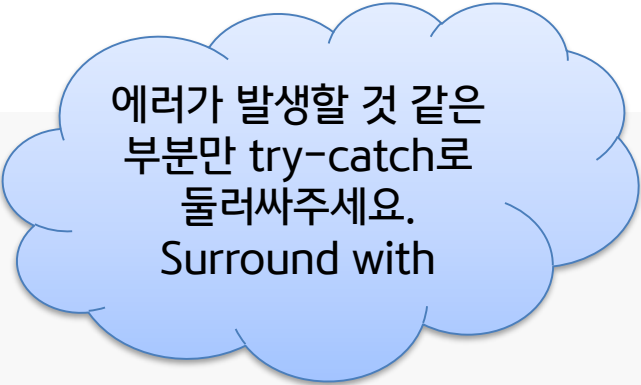
```
package ch11.sec02.exam02;

public class ExceptionHandlingExample {

    public static void main(String[] args) {
        try {
            Class.forName("java.lang.String");
            System.out.println("java.lang.String 클래스가 존재합니다.");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }

        System.out.println();

        try {
            Class.forName("java.lang.String2");
            System.out.println("java.lang.String2 클래스가 존재합니다.");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```



에러가 발생할 것 같은
부분만 try-catch로
둘러싸주세요.
Surround with

```
java.lang.String 클래스가 존재합니다.

java.lang.ClassNotFoundException: java.lang.String2
    at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
    at
    java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:520)
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:375)
    at ch11.sec02.exam02.ExceptionHandlingExample.main(ExceptionHandlingExample.java:16)
```

2 예외 처리 코드

 ExceptionTest2.java

```
package library2;

public class ExceptionTest2 { new *
    public static void main(String[] args) { new *
        Class.forName(className: "java.lang.String");
        System.out.println("java.lang.String 클래스가 존재합니다.");
    }
}
```

Extract Surround //

- try / catch
- try / catch / finally
- if

방법1

방법2

Code Refactor Build Run Tools Git Window Help

- Override Methods... Ctrl+O
- Implement Methods... Ctrl+I
- Delegate Methods...
- Generate... Alt+Insert
- Code Completion >
- Inspect Code...
- Code Cleanup...
- Analyze Code >
- Analyze Stack Trace or Thread Dump...
- Insert Live Template... Ctrl+J
- Save as Live Template...
- Surround With... Ctrl+Alt+T**
- Unwrap/Remove... Ctrl+Shift+Delete

Surround With

1. if
2. if / else
3. while
4. do / while
5. for
- 6. try / catch**
7. try / finally
8. try / catch / finally
9. synchronized
0. Runnable
- A. { }
- B. <editor-fold...> Comments
- C. region...endregion Comments

3 예외 종류에 따른 처리

✓ 다중 catch로 예외 처리하기

- catch 블록의 예외 클래스는 try 블록에서 발생한 예외의 종류를 말함.
- 해당 타입의 예외가 발생하면 catch 블록이 선택되어 실행
- catch 블록이 여러 개라도 catch 블록은 단 하나만 실행됨

```
try {
```

ArrayIndexOutOfBoundsException 발생

NumberFormatException 발생

```
} catch(ArrayIndexOutOfBoundsException e) {
```

예외 처리1

```
} catch(NumberFormatException e) {
```

예외 처리2

```
}
```

3 예외 종류에 따른 처리

ExceptionHandlingExample.java

```
package ch11.sec03.exam01;

public class ExceptionHandlingExample {
    public static void main(String[] args) {
        String[] array = {"100", "100"};

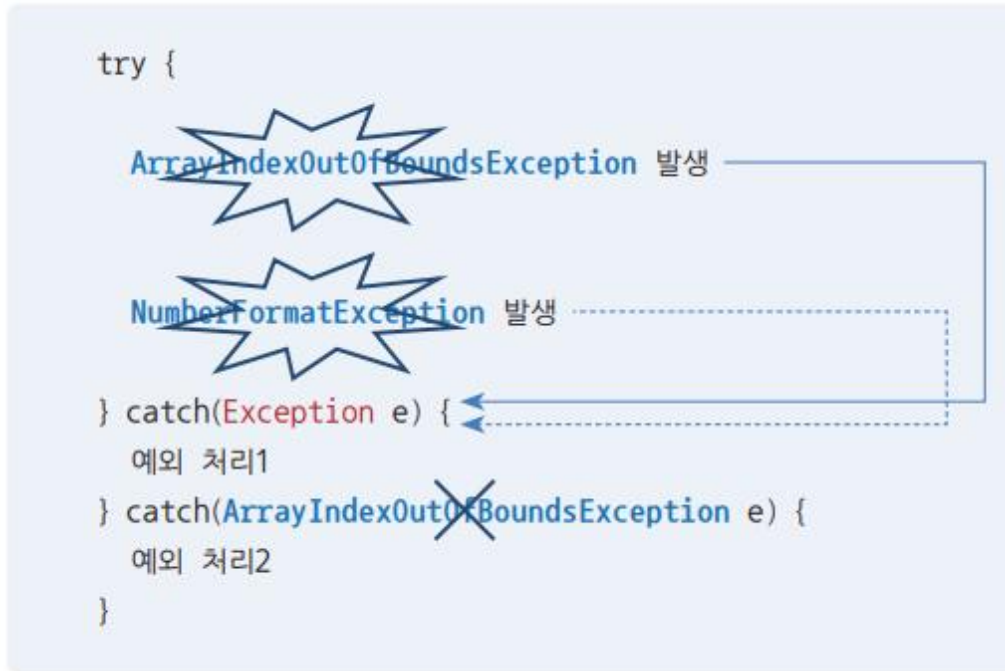
        for(int i=0; i<=array.length; i++) {
            try {
                int value = Integer.parseInt(array[i]);
                System.out.println("array[" + i + "]: " + value);
            } catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("배열 인덱스가 초과됨: " + e.getMessage());
            } catch(NumberFormatException e) {
                System.out.println("숫자로 변환할 수 없음: " + e.getMessage());
            }
        }
    }
}
```

```
array[0]: 100
숫자로 변환할 수 없음: For input string: "100"
배열 인덱스가 초과됨: Index 2 out of bounds for length 2
```

3 예외 종류에 따른 처리

✓ 다중 catch로 예외 처리하기

- 처리해야 할 예외 클래스들이 상속 관계에 있을 때
 - 하위 클래스 catch 블록을 먼저 작성하고 상위 클래스 catch 블록을 나중에 작성해야 함



3 예외 종류에 따른 처리

ExceptionHandlingExample.java

```
package ch11.sec03.exam02;

public class ExceptionHandlingExample {
    public static void main(String[] args) {
        String[] array = {"100", "100"};

        for(int i=0; i<=array.length; i++) {
            try {
                int value = Integer.parseInt(array[i]);
                System.out.println("array[" + i + "]: " + value);
            } catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("배열 인덱스가 초과됨: " + e.getMessage());
            } catch(Exception e) {
                System.out.println("실행에 문제가 있습니다.");
            }
        }
    }
}
```

array[0]: 100
실행에 문제가 있습니다.
배열 인덱스가 초과됨: Index 2 out of bounds for length 2

3 예외 종류에 따른 처리

ExceptionHandlingExample.java

```
package ch11.sec03.exam03;

public class ExceptionHandlingExample {
    public static void main(String[] args) {
        String[] array = {"100", "1oo", null, "200"};

        for(int i=0; i<=array.length; i++) {
            try {
                int value = Integer.parseInt(array[i]);
                System.out.println("array[" + i + "]: " + value);
            } catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("배열 인덱스가 초과됨: " + e.getMessage());
            } catch(NullPointerException | NumberFormatException e) { // 2가지 예외를 동일하게 처리
                System.out.println("데이터에 문제가 있음: " + e.getMessage());
            }
        }
    }
}
```

```
array[0]: 100
데이터에 문제가 있음: For input string: "1oo"
데이터에 문제가 있음: Cannot parse null string
array[3]: 200
배열 인덱스가 초과됨: Index 4 out of bounds for length 4
```

5 예외 처리하기

✓ 예외 처리하기

- 메소드 내부에서 예외 발생 시 throws 키워드 이용해 메소드를 호출한 곳으로 예외 처리하기
- throws는 메소드 선언부 끝에 작성
- 처리할 예외 클래스를 쉼표로 구분해서 나열

```
리턴타입 메소드명(매개변수,...) throws 예외클래스1, 예외클래스2, ... {  
}
```

```
public void method1() {  
    try {  
        method2(); //method2() 호출  
    } catch(ClassNotFoundException e) {  
        System.out.println("예외 처리: " + e.getMessage());  
    }  
}  
  
public void method2() throws ClassNotFoundException {  
    Class.forName("java.lang.String2");  
}
```

호출한 곳에서 예외 처리

 ThrowsExample1.java

```
package ch11.sec05;

public class ThrowsExample1 {
    public static void main(String[] args) {
        try {
            findClass();
        } catch (ClassNotFoundException e) {
            System.out.println("예외 처리: " + e.toString());
        }
    }

    public static void findClass() throws ClassNotFoundException {
        Class.forName("java.lang.String2");
    }
}
```

예외 처리: java.lang.ClassNotFoundException: java.lang.String2

✓ 예외 처리하기

- 나열할 예외 클래스가 많으면 `throws Exception` 또는 `throws Throwable`로 모든 예외 처리하기

```
리턴타입 메소드명(매개변수,...) throws Exception {  
}
```

```
public static void main(String[] args) throws Exception {  
    ...  
}
```

1

```

public class ThrowsExample1 {
    public static void main(String[] args) {
        try {
            findClass();
        } catch (ClassNotFoundException e) {
            System.out.println("예외 처리: " + e.toString());
        }
    }

    public static void findClass() throws ClassNotFoundException {
        Class.forName("java.lang.String2");
    }
}

```

호출한 곳에서
예외 처리호출한 내가 처리하
면 try-catch

2

```

public class ThrowsExample2 {
    public static void main(String[] args) throws Exception {
        findClass();
    }

    public static void findClass() throws ClassNotFoundException {
        Class.forName("java.lang.String2");
    }
}

```

호출한 나를 부른 메
서드로 떠넘기면
throws호출한 메서드에 떠넘겨진 경우에
호출한 메서드는

- 1) 스스로 예외처리를 할 수도
있고,
- 2) 또 떠넘길 수도 있다.

ThrowsExample2.java

```
package ch11.sec05;

public class ThrowsExample2 {
    public static void main(String[] args) throws Exception {
        findClass();
    }

    public static void findClass() throws ClassNotFoundException {
        Class.forName("java.lang.String2");
    }
}
```

```
Exception in thread "main" java.lang.ClassNotFoundException: java.lang.String2
    at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
    at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:520)
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:375)
    at ch11.sec05.ThrowsExample2.findClass(ThrowsExample2.java:9)
    at ch11.sec05.ThrowsExample2.main(ThrowsExample2.java:5)
```

Process finished with exit code 1

6 사용자 정의 예외

✓ 사용자 정의 예외

- 표준 라이브러리에는 없어 직접 정의하는 예외 클래스
- 일반 예외는 `Exception`의 자식 클래스로 선언. 실행 예외는 `RuntimeException`의 자식 클래스로 선언

```
public class XXXException extends [ Exception | RuntimeException ] {  
    public XXXException() {  
    }  
  
    public XXXException(String message) {  
        super(message);  
    }  
}
```

기본 생성자

예외 메시지를 입력받는 생성자

6 사용자 정의 예외

✎ InsufficientException.java

```
package ch11.sec06;

public class InsufficientException extends Exception {
    public InsufficientException() {
    }

    public InsufficientException(String message) {
        super(message);
    }
}
```

개발자가 Exception을 만들 수 있음.

- 기본 생성자, String매개변수 생성자 2개 만들어 줄 것.

InsufficientException.java

```
package ch11.sec06;
```

```
public class InsufficientException extends Exception {
```

```
    public InsufficientException() {
    }
```

```
    public InsufficientException(String message) {
        super(message);
    }
}
```

일반 예외로 선언

두 개의 생성자 선언

6 사용자 정의 예외

✓ 예외 발생시키기

- throw 키워드와 함께 예외 객체를 제공해 사용자 정의 예외를 직접 코드에서 발생시킬 수 있음
- 예외의 원인에 해당하는 메시지를 제공하려면 생성자 매개값으로 전달

```
throw new Exception()
throw new RuntimeException();
throw new InsufficientException();
```

```
throw new Exception("예외메시지")
throw new RuntimeException("예외메시지");
throw new InsufficientException("예외메시지");
```

```
void method() {
  try {
    ...
    throw new Exception("예외메시지");
    ...
  } catch(Exception e) {
    String message = e.getMessage();
  }
}
```

```
void method() throws Exception {
  ...
  throw new Exception("예외메시지");
  ...
}
```

예외를 발생시켜서 던지겠다는 의미
 던져진 Exception은
 1) catch 블록으로 처리할 수 있고,
 2) 메서드를 호출한 곳으로 떠넘길 수 있음.

6 사용자 정의 예외

 Account.java

```
package ch11.sec06;

public class Account {
    private long balance;

    public Account() {}

    public long getBalance() {
        return balance;
    }

    public void deposit(int money) {
        balance += money;
    }

    public void withdraw(int money) throws InsufficientException {
        if(balance < money) {
            throw new InsufficientException("잔고 부족: "+(money-balance)+" 모자람");
        }
        balance -= money;
    }
}
```

잔고보다 인출하는 금액이 많으면
InsufficientException 예외 발생함.
예외 관련 정보는 **withdraw()**를 호출한 메서드로
떠넘김.

6 사용자 정의 예외

 AccountExample.java

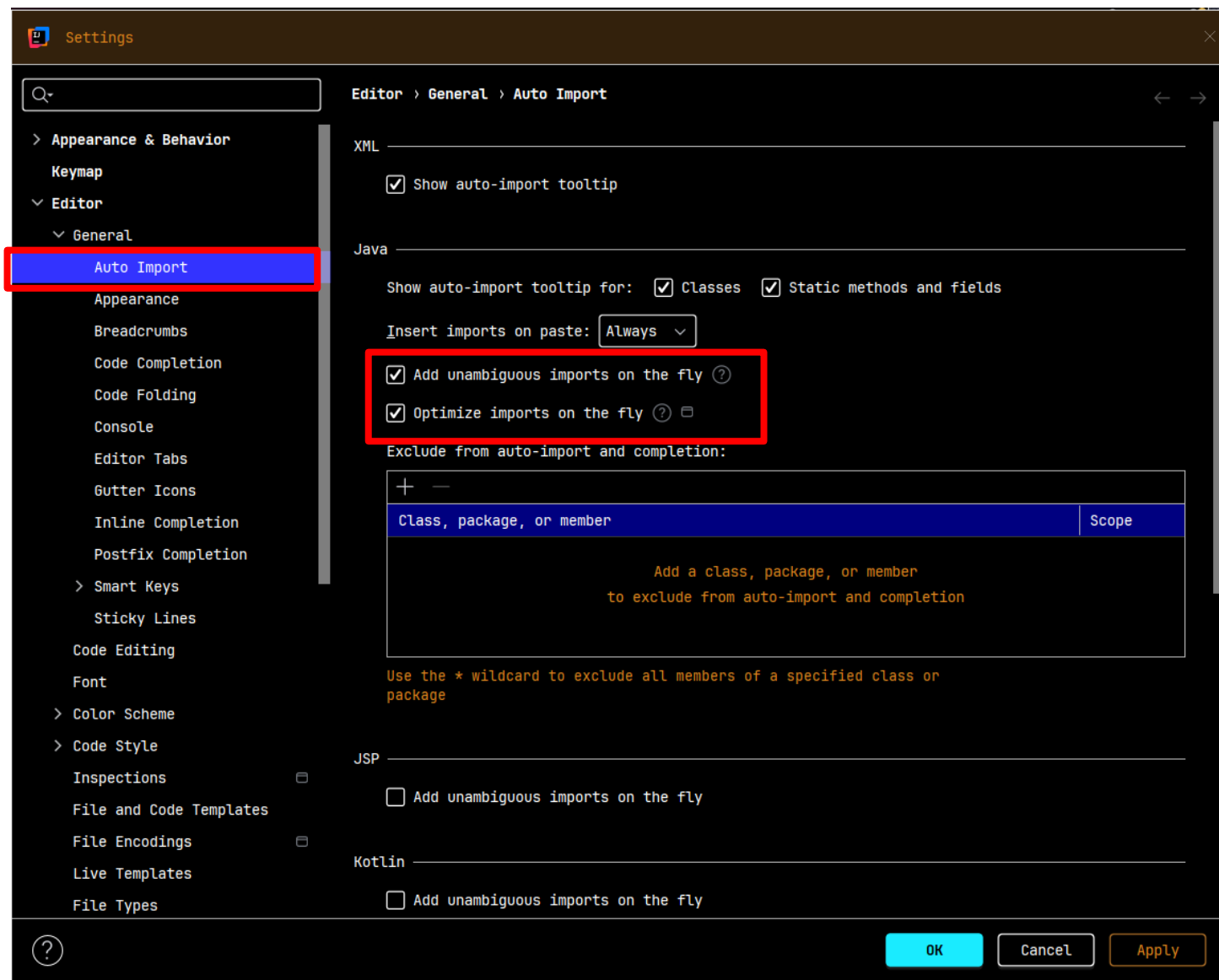
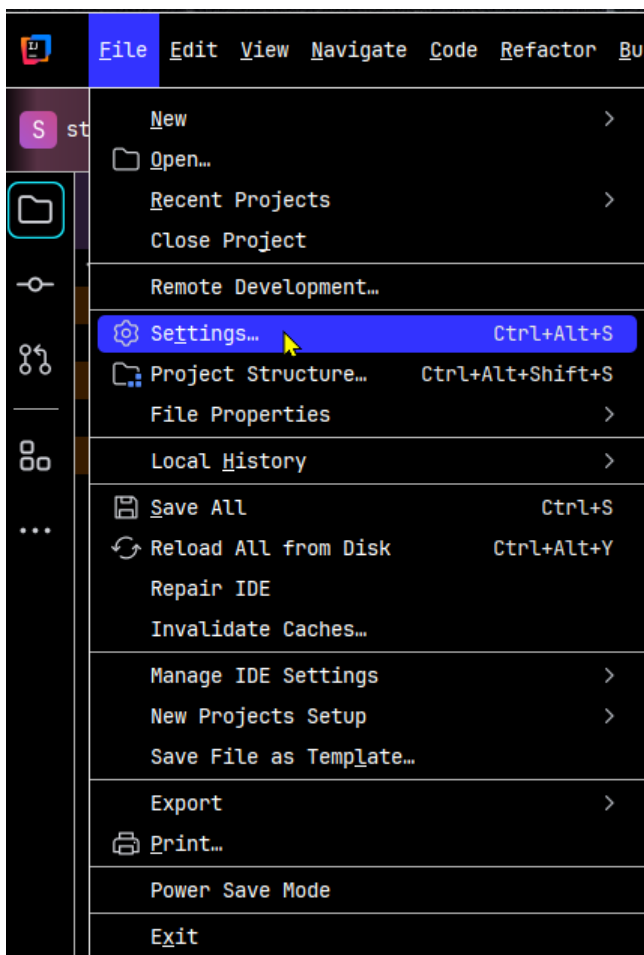
```
package ch11.sec06;

public class AccountExample {
    public static void main(String[] args) {
        Account account = new Account();
        //예금하기
        account.deposit(10000);
        System.out.println("예금액: " + account.getBalance());

        //출금하기
        try {
            account.withdraw(30000);
        } catch (InsufficientException e) {
            String message = e.getMessage();
            System.out.println(message);
        }
    }
}
```

withdraw()로 부터 떠넘겨진
**InsufficientException 예외정보는 catch블록
내에서 처리함.**

예금액: 10000
잔고 부족: 20000 모자람



2025년 상반기 K-디지털 트레이닝

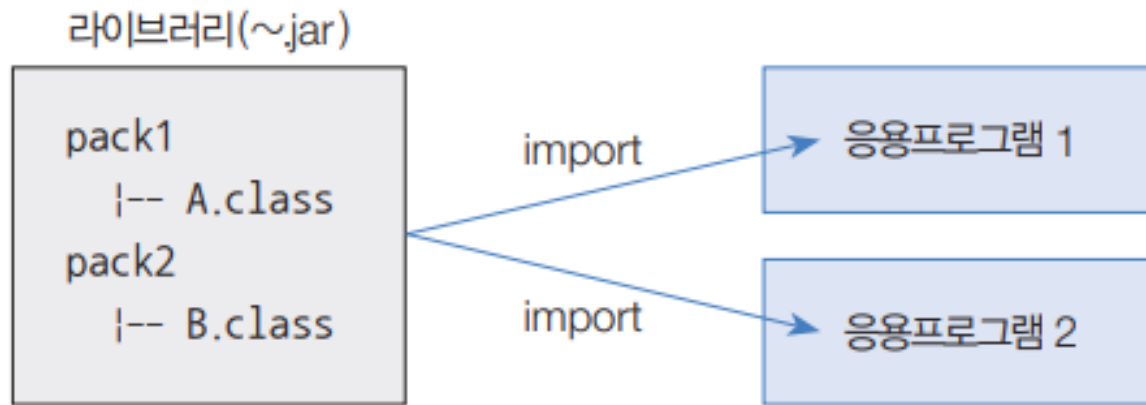
라이브러리와 모듈

[KB] IT's Your Life

1 라이브러리

✓ 라이브러리 추가하기

- 프로그램 개발 시 활용할 수 있는 클래스와 인터페이스들을 모아놓은 것
- 일반적으로 JAR 압축 파일(~.jar) 형태. 클래스와 인터페이스의 바이트코드 파일(~.class)들이 압축



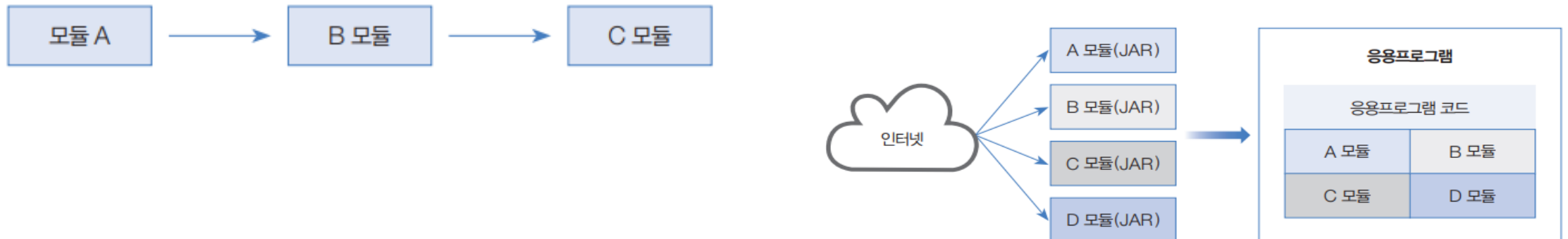
- 라이브러리 JAR 파일을 사용하려면 ClassPath(클래스를 찾기 위한 경로)에 추가
- 콘솔(명령 프롬프트 또는 터미널)에서 프로그램을 실행할 경우: java 명령어를 실행할 때 -classpath로 제공. 또는 CLASSPATH 환경 변수에 경로 추가
- 이클립스 프로젝트에서 실행할 경우: 프로젝트의 Build Path에 추가

✓ 모듈(라이브쇼핑 동영상 → 모듈별 개발 가능)

- 패키지 관리 기능까지 포함된 라이브러리. Java 9부터 지원
- 모듈은 일부 패키지를 은닉하여 접근할 수 없게끔 할 수 있음



- 의존 모듈을 모듈 기술자(module-info.java)에 기술할 수 있어 모듈 간 의존 관계를 파악하기 쉬움
- 대규모 응용프로그램은 기능별로 모듈화해서 개발. 재사용성 및 유지보수에 유리



4 모듈 배포용 JAR 파일

✓ 모듈 배포용 JAR 파일 생성

라이브방송용 모듈
live.jar

쇼핑몰용 모듈
shop.jar

관리자용 모듈
manager.jar

4 모듈 배포용 JAR 파일

✓ 모듈 배포용 JAR 파일 생성

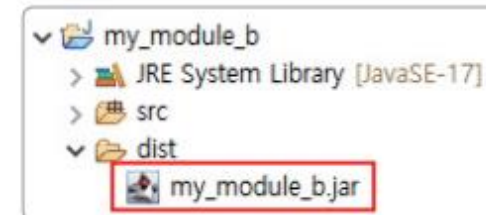
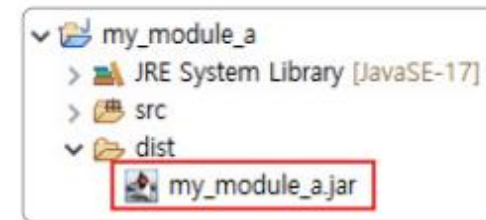
- 다른 모듈에서 쉽게 사용할 수 있게 바이트코드 파일(.class)로 구성된 배포용 JAR 파일을 모듈별로 따로 생성할 수 있음

```

>>> Main.java

1  package app;
2
3  import pack1.A;
4  import pack2.B;
5  import pack3.C;
6
7  public class Main {
8      public static void main(String[] args) {
9          //my_module_a 패키지에 포함된 A 클래스 이용
10         A a = new A();
11         a.method();
12
13         //my_module_a 패키지에 포함된 B 클래스 이용
14         B b = new B();
15         b.method();
16
17         //my_module_b 패키지에 포함된 C 클래스 이용
18         C c = new C();
19         c.method();
20     }
21 }
  
```

my_module_a 모듈에서 가져옴
 my_module_b 모듈에서 가져옴
 my_module_a 모듈의 클래스 이용
 my_module_b 모듈의 클래스 이용

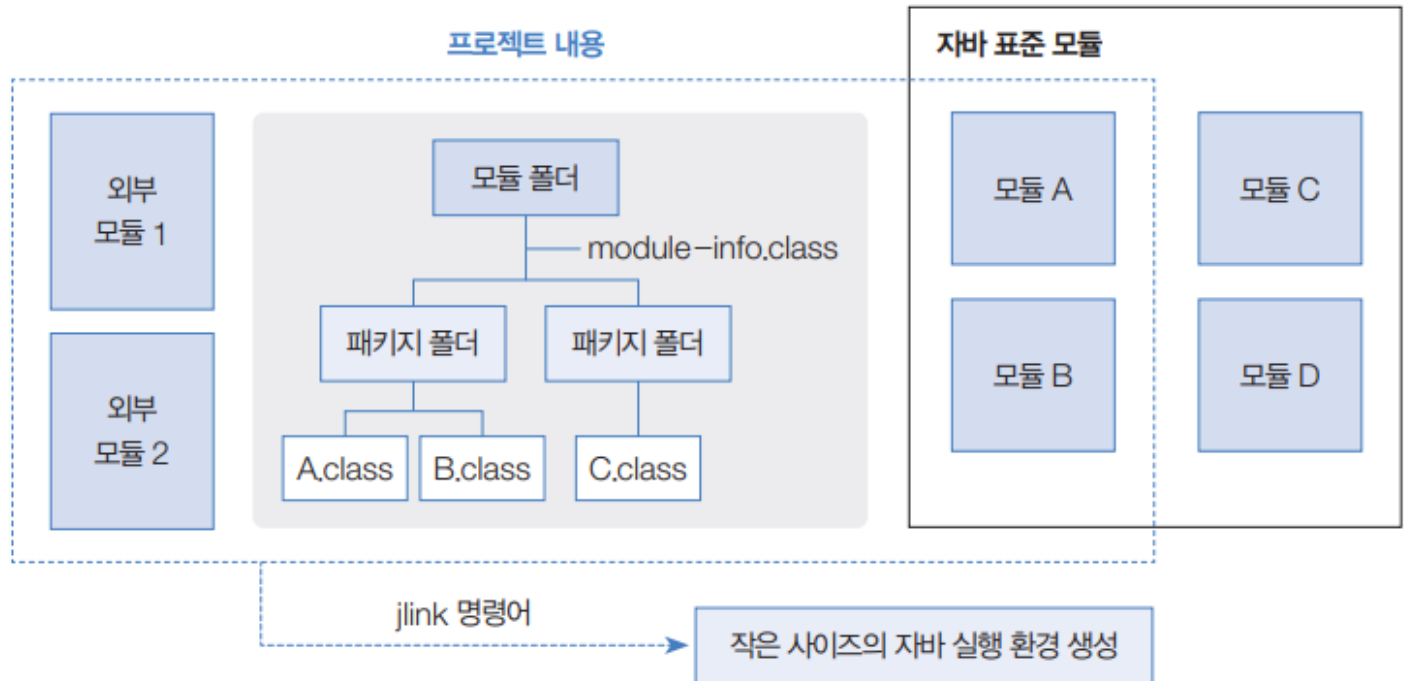
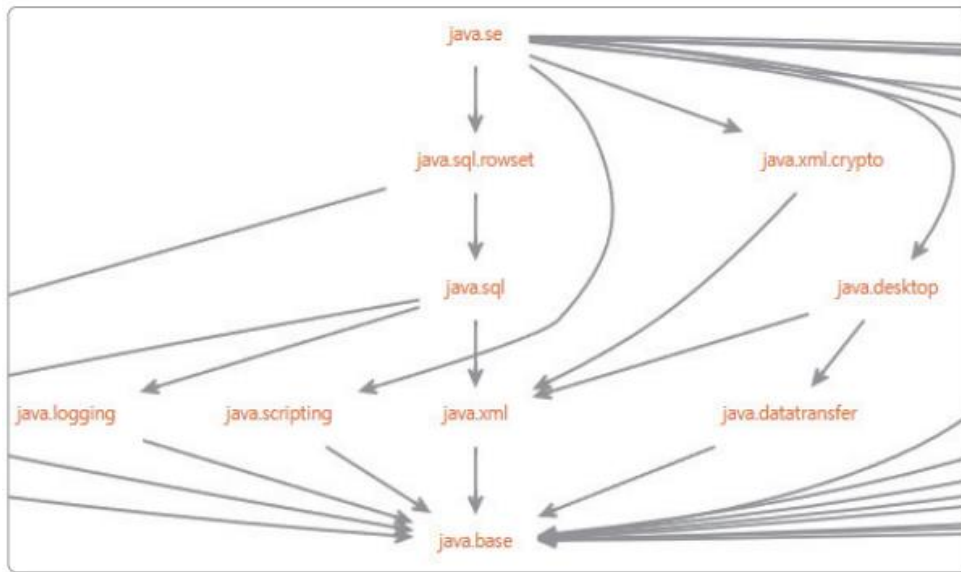


9 자바 표준 라이브러리

✓ 표준 라이브러리

- JDK가 제공하는 표준 라이브러리는 Java 9부터 모듈화됨
- 응용프로그램을 실행하는 데 필요한 모듈만으로 구성된 작은 사이즈의 자바 실행 환경(JRE)
- Java 17의 전체 모듈 그래프(화살표는 모듈간의 의존 관계를 표시)

<https://docs.oracle.com/en/java/javase/17/docs/api/java.se/module-summary.html>



File Edit View Navigate Code Refactor Build Run Tools Git Window Help

study [build] v

Project

- study C:\fullstack\java\study
 - .gradle
 - .idea
 - build
 - classes
 - java
 - main
 - inherit
 - inherit2
 - object
 - object2
 - test1
 - test2
 - generated
 - libs
 - study-1.0-SNAPSHOT.jar
 - tmp
 - gradle
 - out
 - production
 - classes
 - src
 - main
 - inherit
 - HomePhone
 - Phone
 - SmartPhone
 - SmartPhoneUse
 - inherit2

Gradle

- study
 - Tasks
 - build
 - assemble
 - build
 - buildDependents
 - buildNeeded
 - classes
 - clean
 - jar
 - testClasses
 - build setup
 - documentation
 - help

Run study [build] x

```
> Task :compileTestJava NO-SOURCE
> Task :processTestResources NO-SOURCE
> Task :testClasses UP-TO-DATE
> Task :test NO-SOURCE
> Task :check UP-TO-DATE
> Task :build
BUILD SUCCESSFUL in 18s
```

api-gateway

- common-service
 - gradle/wrapper
 - src
 - build.gradle
- elastic
 - .env
 - compose.yml
 - elasticsearch.Dockerfile
 - filebeat.Dockerfile
 - filebeat.yml
 - kibana.yml
 - logstash.conf
- gradle
- lambda-image-resize
 - node_modules
 - index.js
 - package-lock.json
 - package.json
- reserve-service
 - gradle/wrapper
 - src

모듈

모듈

모듈

모듈

2025년 상반기 K-디지털 트레이닝

java.base 모듈

[KB] IT's Your Life

1 API 도큐먼트

✓ API 도큐먼트

- 자바 표준 모듈에서 제공하는 라이브러리를 쉽게 찾아서 사용할 수 있도록 도와주는 문서
- JDK 버전별로 사용할 수 있는 API 도큐먼트:

<https://docs.oracle.com/en/java/javase/index.html>

Java

Home / Java / Java SE

Java Platform, Standard Edition Documentation

Java Platform, Standard Edition (Java SE) helps you develop and deploy Java applications on desktops and servers. Java offers the rich user interface, performance, versatility, portability, and security that today's applications require.

Latest Release

JDK 21

Previous Releases

JDK 20
JDK 19
JDK 18
JDK 17
JDK 16

2 java.base 모듈

✓ java.base

- 모든 모듈이 의존하는 기본 모듈로, 모듈 중 유일하게 requires하지 않아도 사용할 수 있음

패키지	용도
java.lang	자바 언어의 기본 클래스를 제공
java.util	자료 구조와 관련된 컬렉션 클래스를 제공
java.text	날짜 및 숫자를 원하는 형태의 문자열로 만들어 주는 포맷 클래스를 제공
java.time	날짜 및 시간을 조작하거나 연산하는 클래스를 제공
java.io	입출력 스트림 클래스를 제공
java.net	네트워크 통신과 관련된 클래스를 제공
java.nio	데이터 저장을 위한 Buffer 및 새로운 입출력 클래스 제공

2 java.base 모듈

✓ java.lang

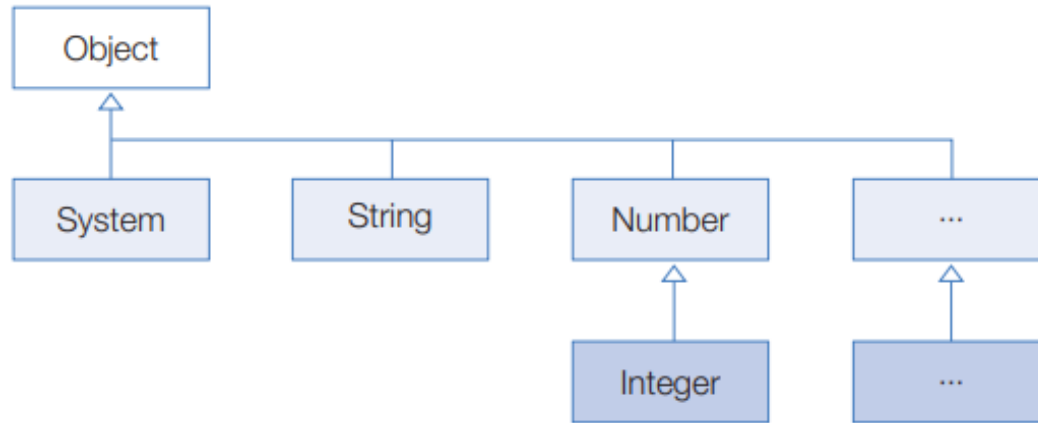
- 자바 언어의 기본적인 클래스를 담고 있는 패키지
- 이 패키지에 있는 클래스와 인터페이스는 import 없이 사용할 수 있음

클래스		용도
Object		- 자바 클래스의 최상위 클래스로 사용
System		- 키보드로부터 데이터를 입력받을 때 사용 - 모니터(콘솔)로 출력하기 위해 사용 - 프로세스를 종료시킬 때 사용 - 진행 시간을 읽을 때 사용 - 시스템 속성(프로퍼티)을 읽을 때 사용
문자열 관련	String	- 문자열을 저장하고 조작할 때 사용
	StringBuilder	- 효율적인 문자열 조작 기능이 필요할 때 사용
	java.util.StringTokenizer	- 구분자로 연결된 문자열을 분리할 때 사용
포장 관련	Byte, Short, Character Integer, Float, Double Boolean	- 기본 타입의 값을 포장할 때 사용 - 문자열을 기본 타입으로 변환할 때 사용
Math		- 수학 계산이 필요할 때 사용
Class		- 클래스의 메타 정보(이름, 구성 멤버) 등을 조사할 때 사용

3 Object 클래스

✓ Object 클래스

- 클래스를 선언할 때 extends 키워드로 다른 클래스를 상속하지 않으면 암시적으로 java.lang.Object 클래스를 상속 → 자바의 모든 클래스는 Object의 자식이거나 자손 클래스



메소드	용도
boolean equals(Object obj)	객체의 번지를 비교하고 결과를 리턴
int hashCode()	객체의 해시코드를 리턴
String toString()	객체 문자 정보를 리턴

3 Object 클래스

✓ 객체 동등 비교

- Object의 equals() 메소드는 객체의 번지를 비교하고 boolean 값을 리턴

```
public boolean equals(Object obj)
```

```
Object obj1 = new Object();  
Object obj2 = obj1;  
boolean result = obj1.equals(obj2);  
boolean result = (obj1 == obj2) ← 결과가 동일
```

3 Object 클래스

✓ 객체 해시코드

- 객체를 식별하는 정수. Object의 hashCode() 메소드는 객체의 메모리 번지를 이용해서 해시코드를 생성하기 때문에 객체마다 다른 정수값을 리턴

```
public int hashCode()
```

- ✓ 자바에서는 힙 영역의 실제 메모리 주소를 직접 출력하거나 조작할 수 없음.
- ✓ 대신에 hashCode()는 객체를 식별하거나 비교하기 위한 '고유값'**을 제공
- ☞ 이유는 보안, 안정성, 이식성, GC(Garbage Collection) 때문

자바는 "안전하고 이식 가능한 언어"를 목표로 설계되었기 때문에,
C나 C++처럼 직접 메모리 주소(포인터)를 다루는 방식은 **의도적으로 금지**

3 Object 클래스

✓ 객체 문자 정보

- 객체를 문자열로 표현한 값. Object의 toString() 메소드는 객체의 문자 정보를 리턴
- 기본적으로 Object의 toString() 메소드는 '클래스명@16진수해시코드'로 구성된 문자열을 리턴

```
Object obj = new Object();  
System.out.println(obj.toString());
```



java.lang.Object@de6ced

3 Object 클래스

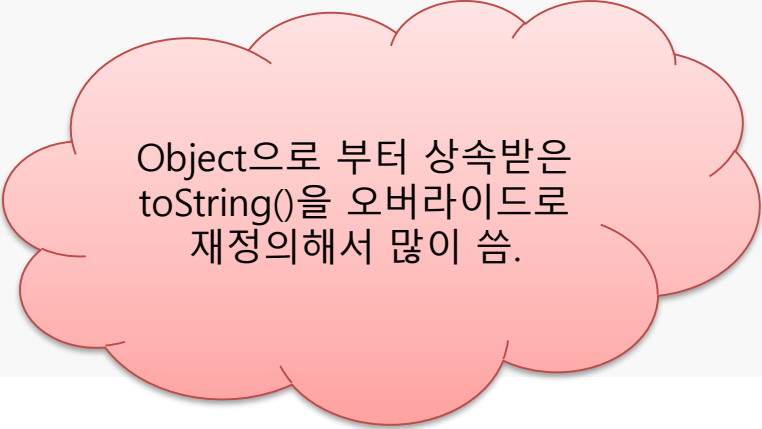
SmartPhone.java

```
package ch12.sec03.exam03;
```

```
public class SmartPhone {  
    private String company;  
    private String os;
```

```
    public SmartPhone(String company, String os) {  
        this.company = company;  
        this.os = os;  
    }
```

```
    @Override  
    public String toString() {  
        return company + ", " + os;  
    }  
}
```



Object으로 부터 상속받은
toString()을 오버라이드로
재정의해서 많이 씀.

3 Object 클래스

ToStringExample.java

```
package ch12.sec03.exam03;

public class ToStringExample {
    public static void main(String[] args) {
        SmartPhone myPhone = new SmartPhone("삼성전자", "안드로이드");

        //String strObj = myPhone.toString();
        //System.out.println(strObj);

        System.out.println(myPhone); //toString()이 오버라이드되어 주소 대신 재정의한 문자열이 출력
    }
}
```

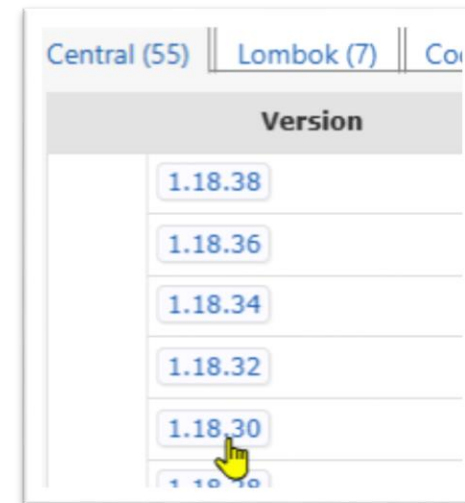
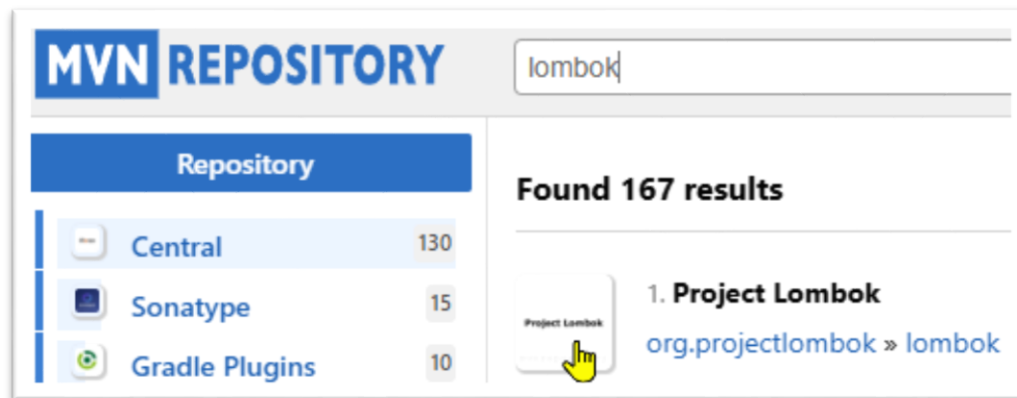
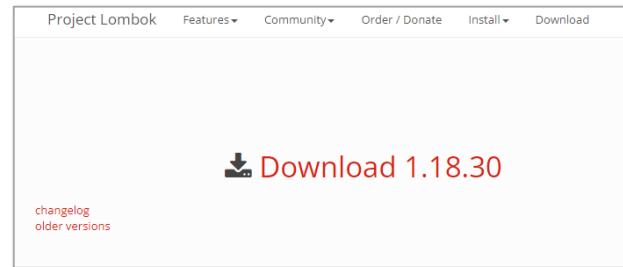
삼성전자, 안드로이드
삼성전자, 안드로이드

3 Object 클래스

✓ 롬복 사용하기(주요 함수 자동 생성해주는 라이브러리)

- DTO 클래스(가방역할)를 작성할 때 Getter, Setter, hashCode(), equals (), toString() 메소드를 자동 생성
- 필드가 final이 아니며, 값을 읽는 Getter는 getXxx(또는 isXxx)로, 값을 변경하는 Setter는 setXxx로 생성됨

- <https://projectlombok.org/download>
- <https://mvnrepository.com/>



3 Object 클래스

Files pom (1 KB) jar (1.9 MB) View All

Repositories Central Auxilor NeetGames Public SpaceIO

Ranking #13 in MvnRepository (See Top Artifacts)
#1 in Code Generators

Used By 28,109 artifacts

Note: There is a new version for this artifact

New Version	1.18.38
--------------------	---------

Maven Gradle SBT Mill Ivy Grape Leiningen Buildr

Scope: Compile Format: Groovy Short

```
// https://mvnrepository.com/artifact/org.projectlombok/lombok
implementation 'org.projectlombok:lombok:1.18.30'
```

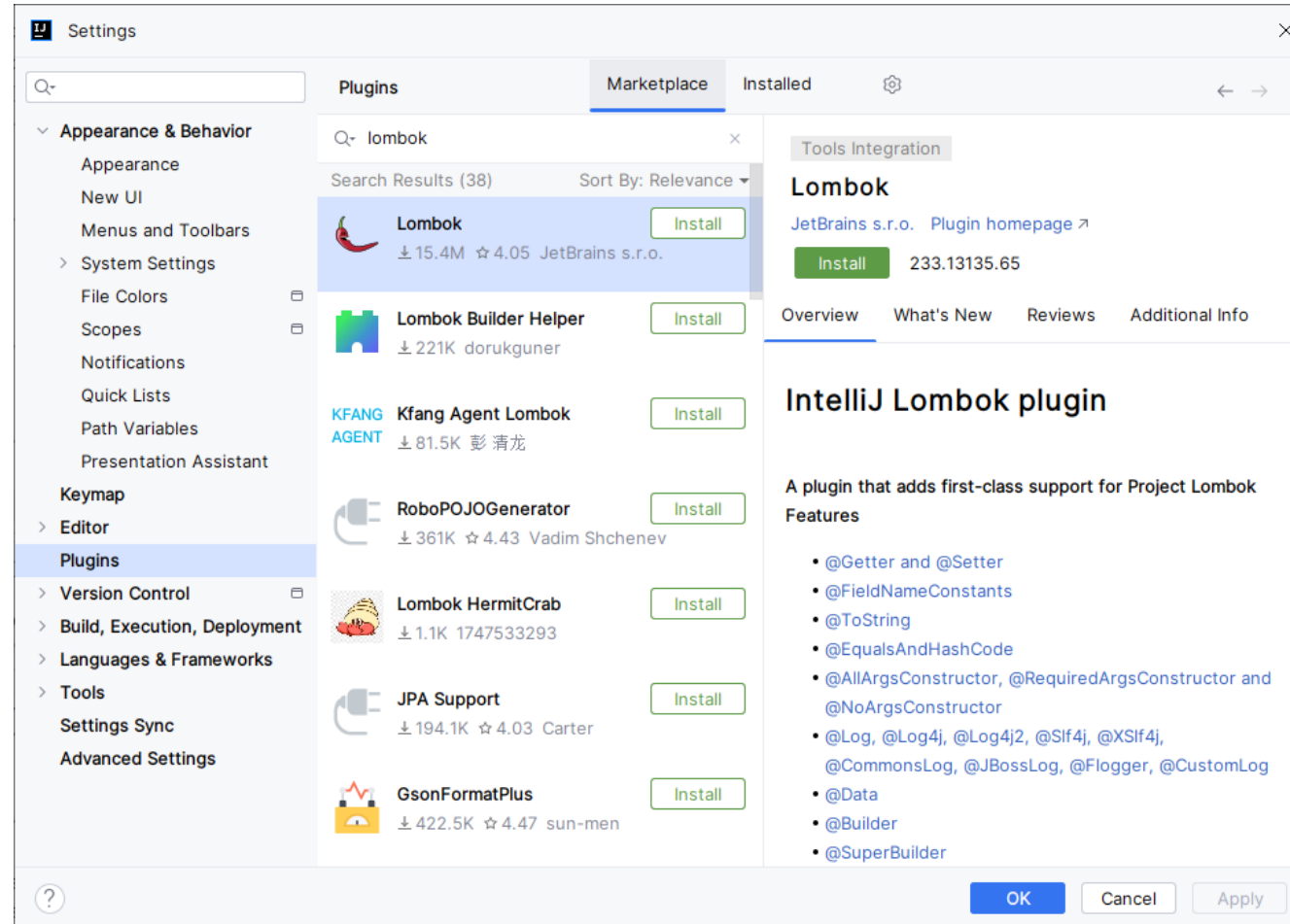
Jar파일 다운로드 후 intellij에 수동 설정

복사해서 intellij에 넣으면
자동으로 설정됨.

3 Object 클래스

✓ 롬복 플러그 인 설치

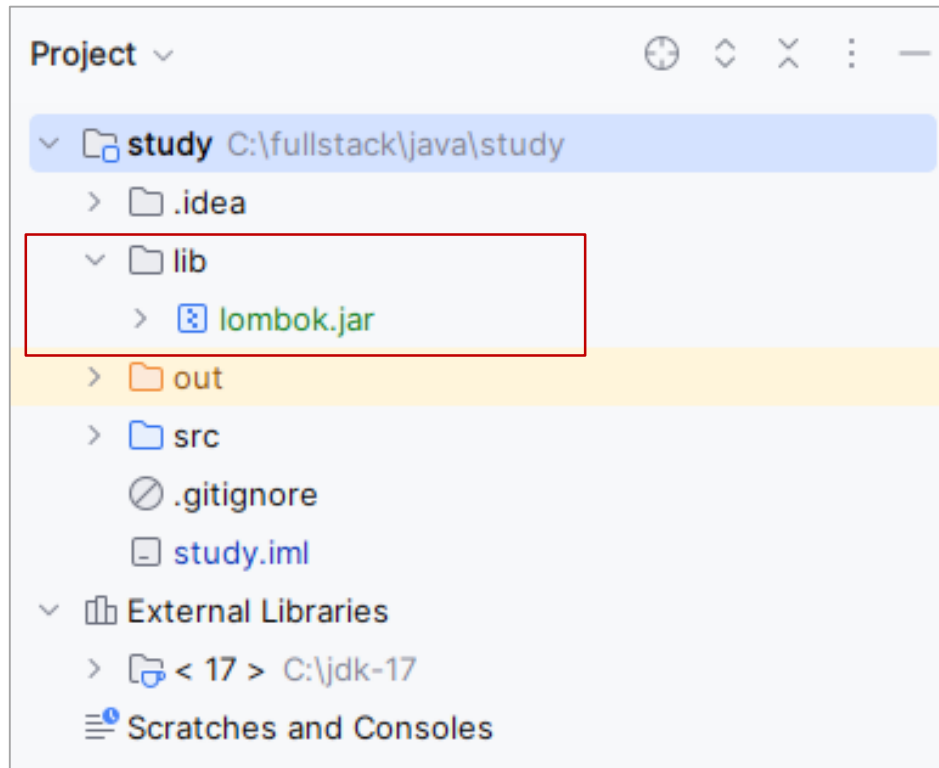
- File → Settings → Plugins → Marketplace
- lombok 검색



3 Object 클래스

✓ 롬복 라이브러리 사용하기

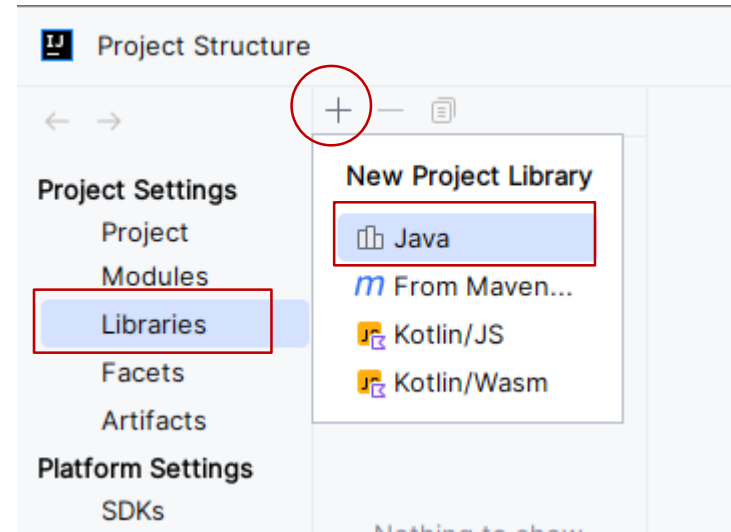
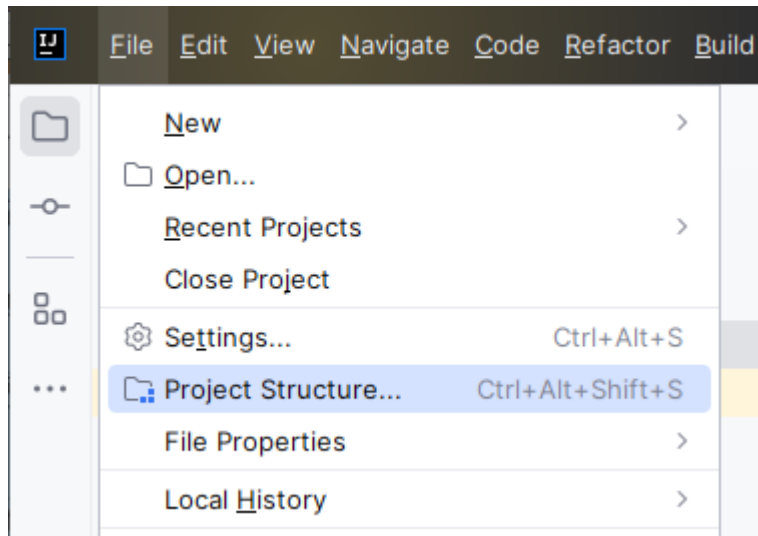
- 순서 1) lib 디렉토리 생성 후 lombok.jar 추가



3 Object 클래스

✓ 롬복 라이브러리 사용하기

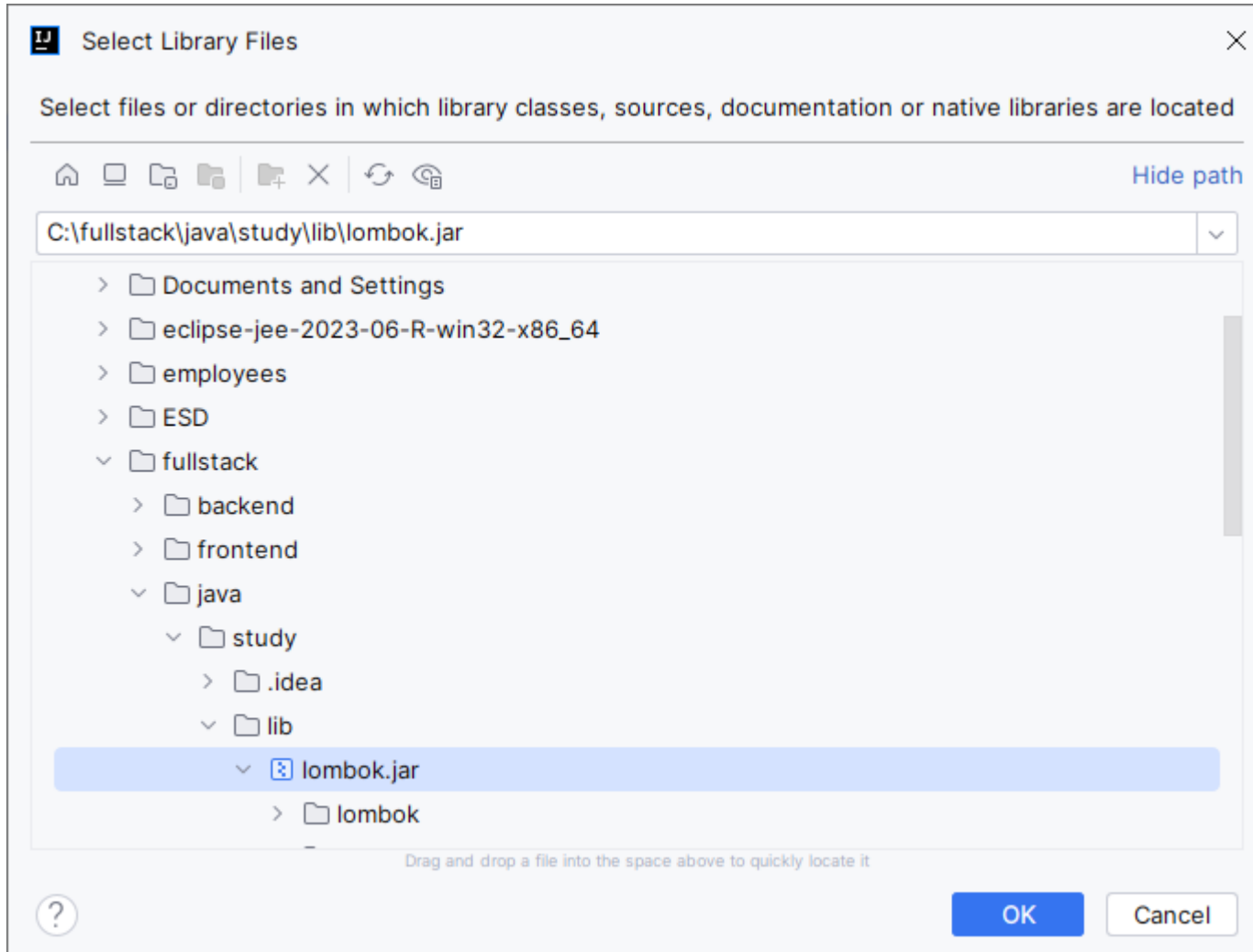
- 순서 2) 프로젝트에 lombok.jar을 라이브러리로 등록
 - File > Project Structure



3 Object 클래스

✓ 롬복 라이브러리 사용하기

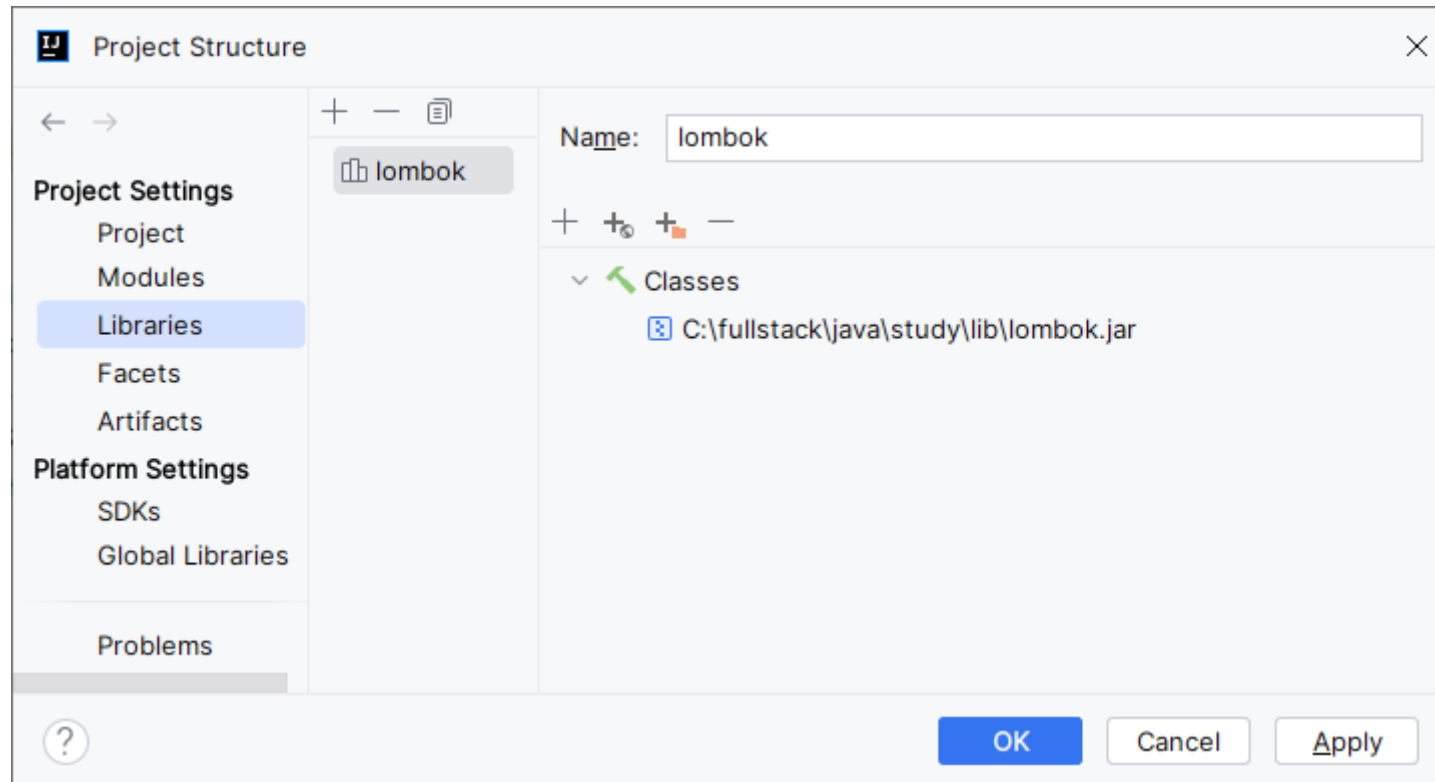
- 방법 2) 프로젝트에 lombok.jar을 라이브러리로 등록



3 Object 클래스

✓ 롬복 라이브러리 사용하기

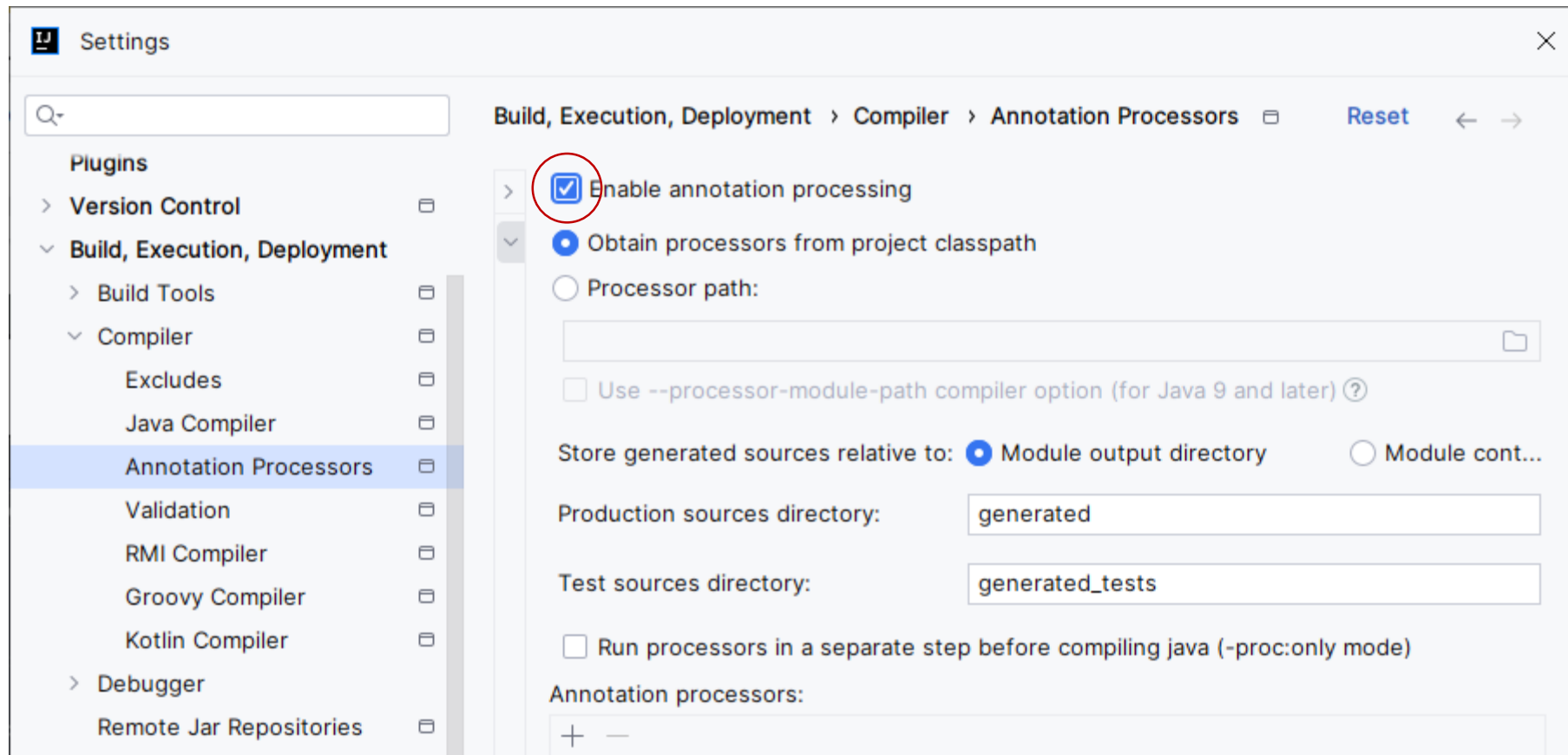
- 방법 2) 프로젝트에 lombok.jar을 라이브러리로 등록



3 Object 클래스

✓ 롬복 라이브러리 사용하기

- Annotation Processors 설정
- File > Settings > Build, Execution, Deployment > Compiler > Annotation Processors



3 Object 클래스

✓ 롬복 라이브러리 사용하기

○ 롬복 어노테이션

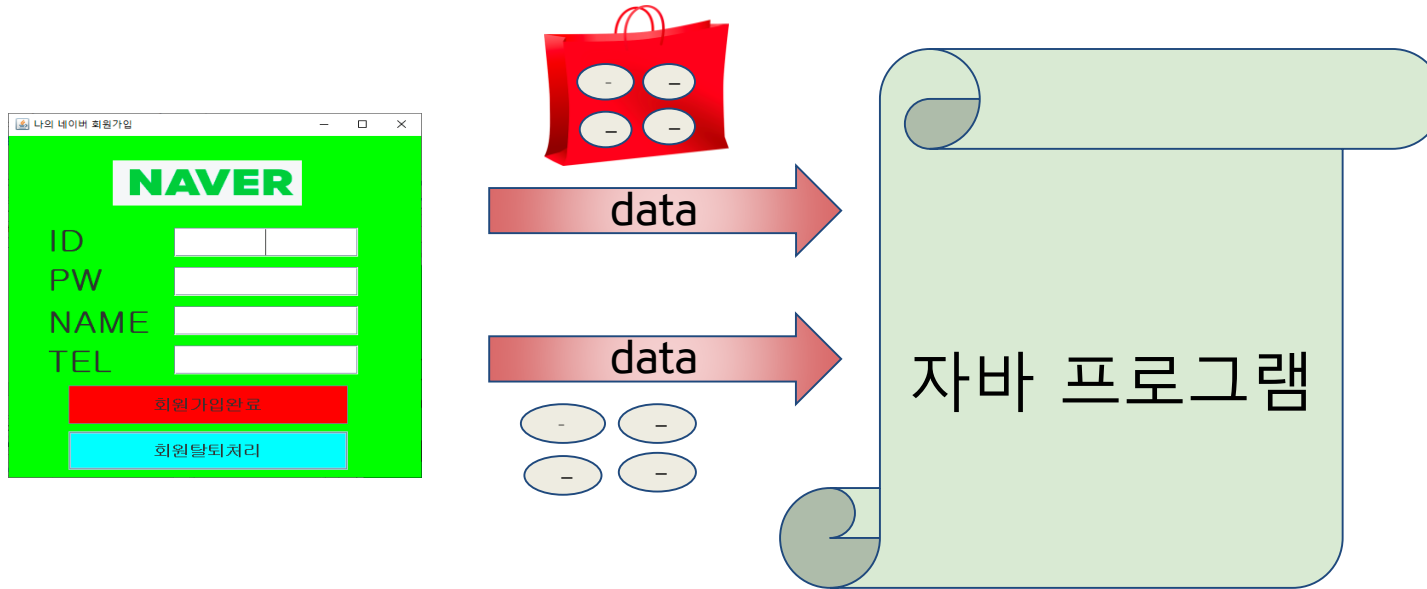
어노테이션	설명
@NoArgsConstructor	기본(매개변수가 없는) 생성자 포함
@AllArgsConstructor	모든 필드를 초기화시키는 생성자 포함
@RequiredArgsConstructor	기본적으로 매개변수가 없는 생성자 포함. 만약 final 또는 @NonNull이 붙은 필드가 있다면 이 필드만 초기화시키는 생성자 포함
@Getter	Getter 메소드 포함
@Setter	Setter 메소드 포함
@EqualsAndHashCode	equals()와 hashCode() 메소드 포함
@ToString	toString() 메소드 포함

○ @Data

- @RequiredArgsConstructor, @Getter, @Setter, @EqualsAndHashCode, @ToString 합친 것

DTO(Data Transfer Object, VO, Value Object)

- 많은 양의 데이터를 전달할 목적으로 사용하는 **가방 역할의 클래스**
- 값들만 넣어서 전달하고, 전달된 값들을 꺼내어 사용하는 역할
- 자바에서는 여러 데이터를 전달할 때 하나씩 전달하지 않고, 데이터들을 넣어서 전달할 클래스(가방역할)을 만들어 사용한다.



3 Object 클래스

Student.java

```
package ch12.sec03.exam02;
```

```
public class Student {
    private int no;
    private String name;

    public Student(int no, String name) {
        this.no = no;
        this.name = name;
    }
}
```

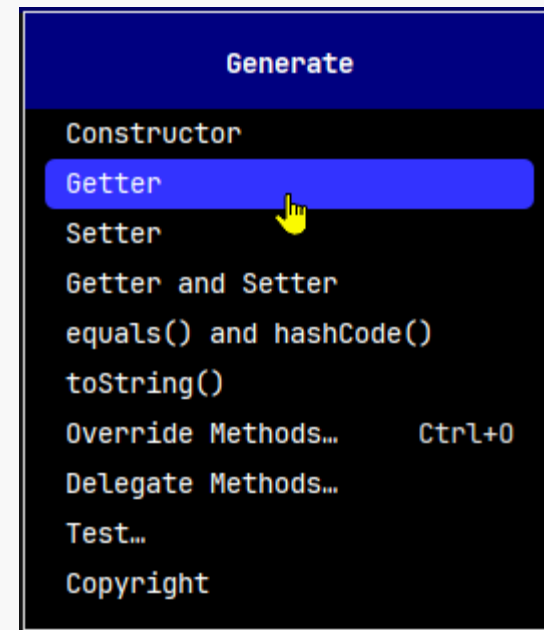
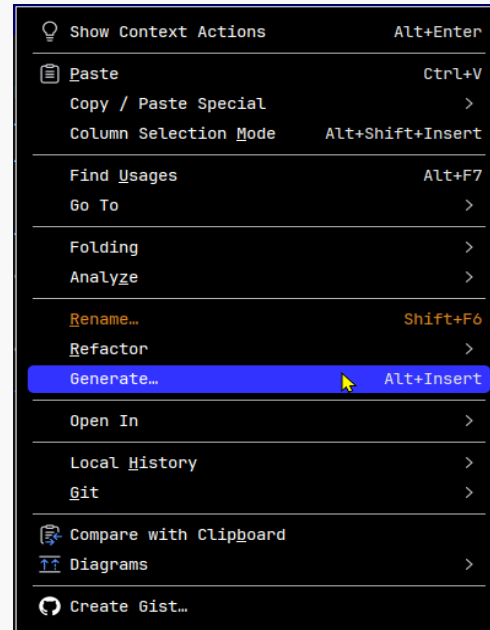
```
public int getNo() {
    return no;
}

public String getName() {
    return name;
}

public void setNo(int no) {
    this.no = no;
}

public void setName(String name) {
    this.name = name;
}
}
```

- 필드 하나당 값을 넣거나 꺼내는 메서드를 만들어 많이 사용함.
 - 넣는 메서드 : setter, set필드명(매개변수) 메서드 형태
 - 꺼내는 메서드 : getter, get필드명() 메서드 형태



3 Object 클래스

Member.java

```
package ch12.sec03.exam05;
```

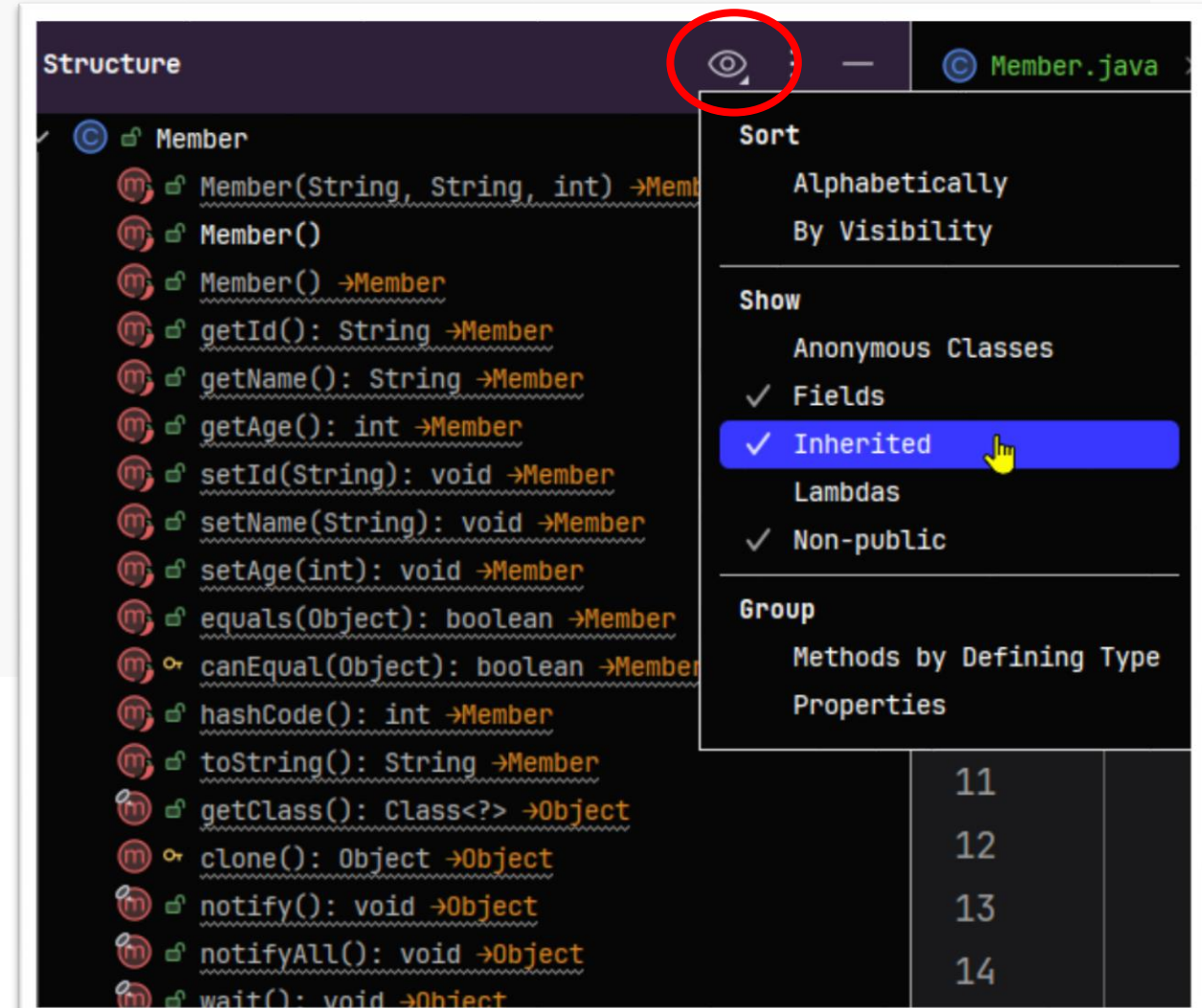
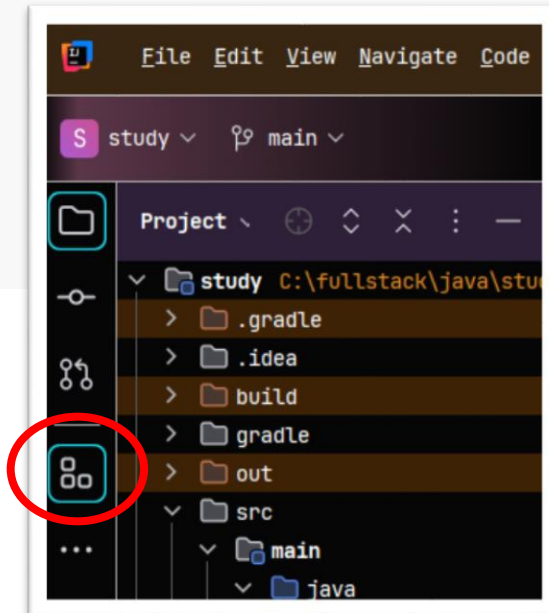
```
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
```

@Data

@NoArgsConstructor

@AllArgsConstructor

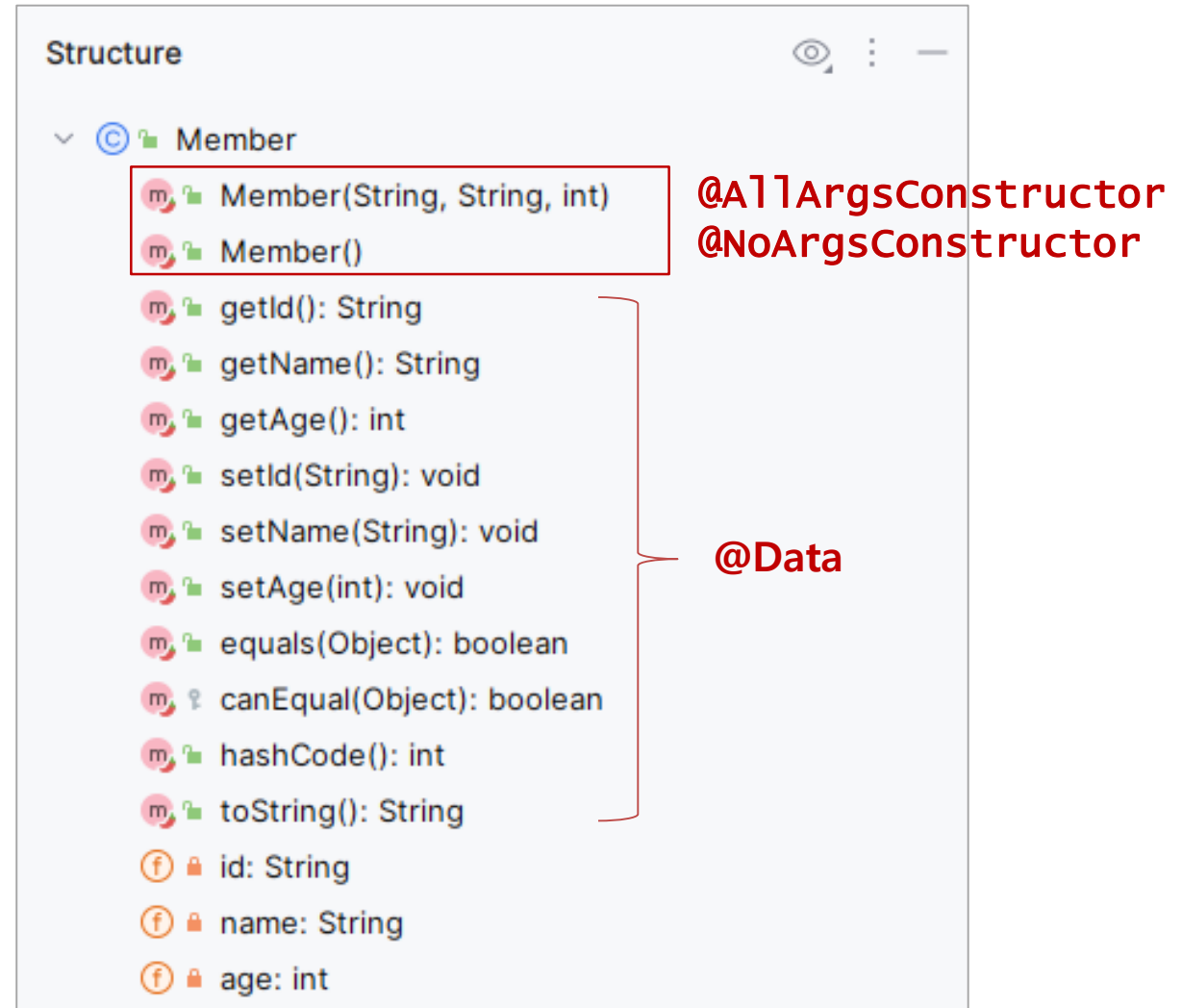
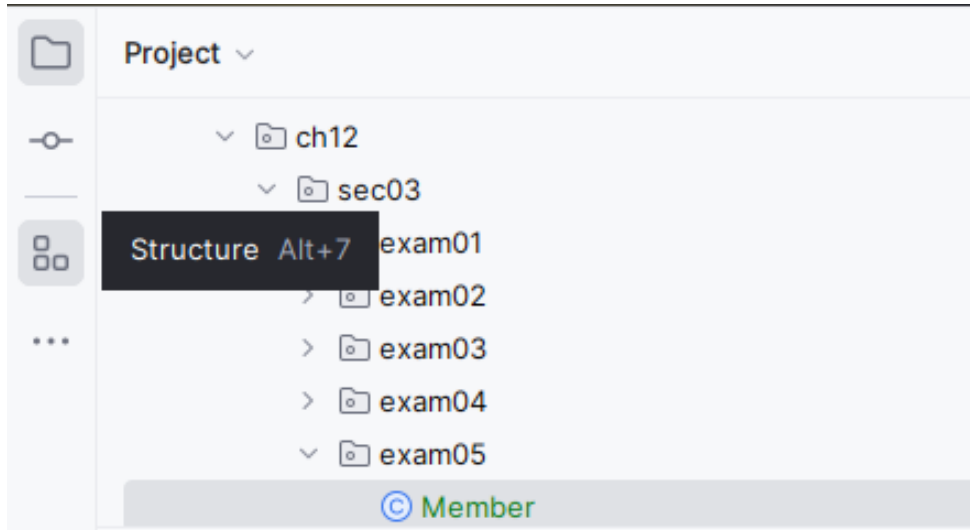
```
public class Member {
    private String id;
    private String name;
    private int age;
}
```



3 Object 클래스

✓ 생성된 코드 확인

- Structure 아이콘 클릭 또는 Alt + 7



3 Object 클래스

Member.java

```
package ch12.sec03.exam05;
```

```
import lombok.Data;  
import lombok.NonNull;
```

@Data

```
public class Member {  
    private String id;  
    @NonNull // 필수 항목  
    private String name;  
    private int age;  
}
```

→ @RequiredArgsConstructor에 의해 생성자 추가됨

Structure

Member

Member(String)

getId(): String

getName(): String

getAge(): int

setId(String): void

setName(String): void

setAge(int): void

equals(Object): boolean ↑Object

canEqual(Object): boolean

hashCode(): int ↑Object

toString(): String ↑Object

id: String

name: String

age: int

4 System 클래스

✓ System 클래스

- System 클래스의 정적 static 필드와 메소드를 이용하면 프로그램 종료, 키보드 입력, 콘솔(모니터) 출력, 현재 시간 읽기, 시스템 프로퍼티 읽기 등이 가능

정적 멤버		용도
필드	out	콘솔(모니터)에 문자 출력
	err	콘솔(모니터)에 에러 내용 출력
	in	키보드 입력
메소드	exit(int status)	프로세스 종료
	currentTimeMillis()	현재 시간을 밀리초 단위의 long 값으로 리턴
	nanoTime()	현재 시간을 나노초 단위의 long 값으로 리턴
	getProperty()	운영체제와 사용자 정보 제공
	getenv()	운영체제의 환경 변수 정보 제공

4 System 클래스

ErrExample.java 콘솔 출력

```
package ch12.sec04;

public class ErrExample {

    public static void main(String[] args) {
        try {
            int value = Integer.parseInt("1oo");
        } catch (NumberFormatException e) {
            System.err.println("[에러 내용]");
            System.err.println(e.getMessage());
        }
    }
}
```

[에러 내용]
For input string: "1oo"

4 System 클래스

✓ 키보드 입력

○ System.in 객체 이용

```
int keyCode = System.in.read();
```

○ 키 코드

0 = 48	A = 65	N = 78	a = 97	n = 110	[Windows]
1 = 49	B = 66	O = 79	b = 98	o = 111	Enter = 13, 10
2 = 50	C = 67	P = 80	c = 99	p = 112	
3 = 51	D = 68	Q = 81	d = 100	q = 113	[macOS]
4 = 52	E = 69	R = 82	e = 101	r = 114	Enter = 10
5 = 53	F = 70	S = 83	f = 102	s = 115	
6 = 54	G = 71	T = 84	g = 103	t = 116	
7 = 55	H = 72	U = 85	h = 104	u = 117	
8 = 56	I = 73	V = 86	i = 105	v = 118	
9 = 57	J = 74	W = 87	j = 106	w = 119	
	K = 75	X = 88	k = 107	x = 120	
	L = 76	Y = 89	l = 108	y = 121	
	M = 77	Z = 90	m = 109	z = 122	

4 System 클래스

ErrExample.java 키보드 입력

```
package ch12.sec04;

public class InExample {
    public static void main(String[] args) throws Exception {
        int speed = 0;
        int keyCode = 0;

        while(true) {
            //Enter 키를 읽지 않았을 경우에만 실행
            if(keyCode != 13 && keyCode != 10) {
                if (keyCode == 49) {           //숫자 1 키를 읽었을 경우
                    speed++;
                } else if (keyCode == 50) {    //숫자 2 키를 읽었을 경우
                    speed--;
                } else if (keyCode == 51) {    //숫자 3 키를 읽었을 경우
                    break;
                }
                System.out.println("-----");
                System.out.println("1. 증속 | 2. 감속 | 3. 중지");
                System.out.println("-----");
                System.out.println("현재 속도= " + speed);
                System.out.print("선택: ");
            }
        }
    }
}
```

4 System 클래스

 ErrExample.java

키보드 입력

```
//키를 하나씩 읽음
keyCode = System.in.read();
} // end of while

System.out.println("프로그램 종료");
}
```

1. 증속 | 2. 감속 | 3. 중지

현재 속도= 0
선택: 1

1. 증속 | 2. 감속 | 3. 중지

현재 속도= 1
선택: 2

1. 증속 | 2. 감속 | 3. 중지

현재 속도= 0
선택: 3
프로그램 종료

4 System 클래스

✓ 프로세스 종료

○ 프로세스 process

- 운영체제는 실행 중인 프로그램을 프로세스로 관리
- 자바 프로그램을 시작하면 JVM 프로세스가 생성
 - 이 프로세스가 main() 메서드 호출

○ System.exit() 메서드로 현재 프로세스를 강제 종료 가능

- System.exit(int status)
- 매개변수로 종료 상태 코드 지정
 - 0: 정상 종료(관례)
 - 그 외 에러 상태를 나타내는 정수 지정

4 System 클래스

ExitExample.java

```
package ch12.sec04;

public class ExitExample {
    public static void main(String[] args) {
        for(int i=0; i<10; i++) {
            //i값 출력
            System.out.println(i);
            if(i == 5) {
                //JVM 프로세스 종료
                System.out.println("프로세스 강제 종료");
                System.exit(0);
            }
        }
    }
}
```

```
0
1
2
3
4
5
프로세스 강제 종료
```


4 System 클래스

✓ 진행 시간 읽기

메소드	용도
<code>long currentTimeMillis()</code>	1/1000 초 단위로 진행된 시간을 리턴
<code>long nanoTime()</code>	1/10 ⁹ 초 단위로 진행된 시간을 리턴

4 System 클래스

MeasureRunTimeExample.java

```
package ch12.sec04;

public class MeasureRunTimeExample {
    public static void main(String[] args) {
        long time1 = System.nanoTime();
        int sum = 0;
        for(int i=1; i<=1000000; i++) {
            sum += i;
        }
        long time2 = System.nanoTime();

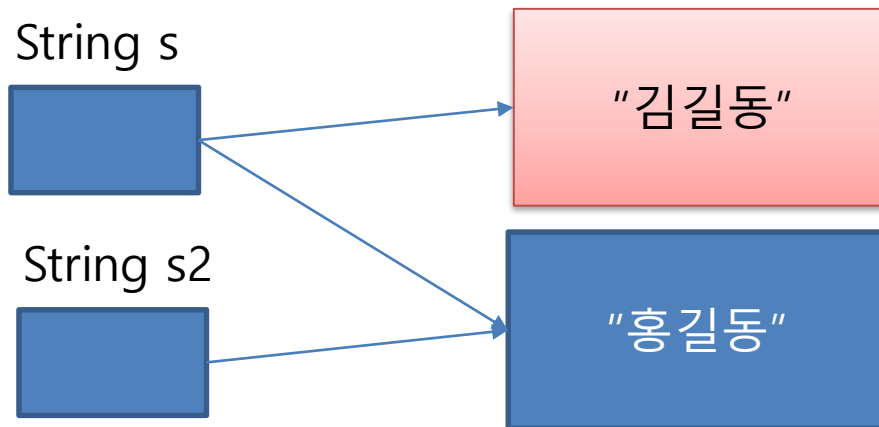
        System.out.println("1~1000000까지의 합: " + sum);
        System.out.println("계산에 " + (time2-time1) + " 나노초가 소요되었습니다.");
    }
}
```

서버 성능 체크시
소요 시간을 계산
할 때 많이 사용함.

1~1000000까지의 합: 1784293664
계산에 4161100 나노초가 소요되었습니다.

☑ 문자열 관련 주요 클래스

클래스	설명
String	문자열을 저장하고 조작할 때 사용
StringBuilder	효율적인 문자열 조작 기능이 필요할 때 사용
StringTokenizer	구분자로 연결된 문자열을 분리할 때 사용



//String은 값이 변경되면 기존 것은 놔두고 새로운 저장공간을 만들어 변경된 값을 넣음.
 //값이 변경될 때마다 새 저장공간을 만들므로 값이 변경되는 경우 String을 사용하면 성능저하가 발생 → StringBuilder사용해야함.

```

String s = "홍길동";
String s2 = "홍길동";
// s == s2(true, 주소가 동일O)
s = "김길동";
// s == s2(false, 주소가 동일X)
  
```

✓ String 클래스

- String 클래스는 문자열을 저장하고 조작할 때 사용
- 문자열 리터럴은 자동으로 String 객체로 생성. String 클래스의 다양한 생성자를 이용해서 직접 객체를 생성할 수도 있음

```
//기본 문자셋으로 byte 배열을 디코딩해서 String 객체로 생성  
String str = new String(byte[] bytes);
```

```
//특정 문자셋으로 byte 배열을 디코딩해서 String 객체로 생성  
String str = new String(byte[] bytes, String charsetName);
```

- 한글 1자를 UTF-8로 인코딩하면 3바이트가 되고, EUC-KR로 인코딩하면 2바이트가 됨

BytesToStringExample.java

```
package ch12.sec05;

import java.util.Arrays;

public class BytesToStringExample {
    public static void main(String[] args) throws Exception {
        String data = "자바";

        //String -> byte 배열(기본: UTF-8 인코딩)
        byte[] arr1 = data.getBytes();
        //byte[] arr1 = data.getBytes("UTF-8");
        System.out.println("arr1: " + Arrays.toString(arr1));

        //byte 배열 -> String(기본: UTF-8 디코딩)
        String str1 = new String(arr1);
        //String str1 = new String(arr1, "UTF-8");
        System.out.println("str1: " + str1);
    }
}
```

TCP/UDP
네트워크 통신에 사용

네트워크는 바이트 단위로 전송
→ 자바에서 문자열, 이미지, 파일 등으로 생성

BytesToStringExample.java

```
//String -> byte 배열(EUC-KR 인코딩)
byte[] arr2 = data.getBytes("EUC-KR");
System.out.println("arr2: " + Arrays.toString(arr2));

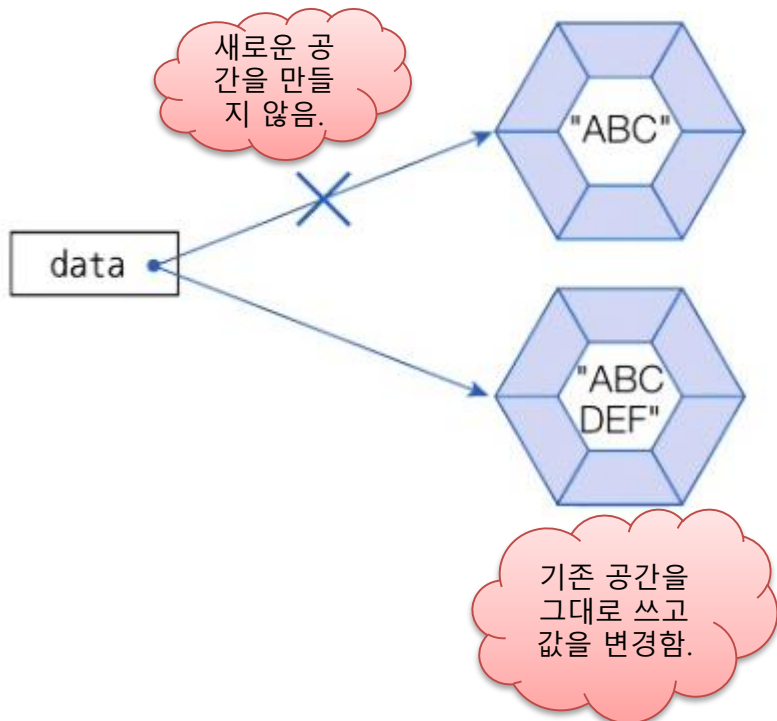
//byte 배열 -> String(기본: UTF-8 디코딩)
String str2 = new String(arr2, "EUC-KR");
System.out.println("str2: " + str2);
}
```

```
arr1: [-20, -98, -112, -21, -80, -108]
str1: 자바
arr2: [-64, -38, -71, -39]
str2: 자바
```

✓ StringBuilder 클래스

- **잡은 문자열 변경 작업을 해야 한다면 String보다는 StringBuilder가 좋음**
- StringBuilder는 내부 버퍼에 문자열을 저장해두고 그 안에서 추가, 수정, 삭제 작업을 하도록 설계

```
String data = "ABC";
data += "DEF";
```



리턴 타입	메소드(매개변수)	설명
StringBuilder	append(기본값 문자열)	문자열을 끝에 추가
StringBuilder	insert(위치, 기본값 문자열)	문자열을 지정 위치에 추가
StringBuilder	delete(시작 위치, 끝 위치)	문자열 일부를 삭제
StringBuilder	replace(시작 위치, 끝 위치, 문자열)	문자열 일부를 대체
String	toString()	완성된 문자열을 리턴

StringBuilderExample.java

```
package ch12.sec05;

public class StringBuilderExample {
    public static void main(String[] args) {
        String data = new StringBuilder()
            .append("DEF")
            .insert(0, "ABC")
            .delete(3, 4)
            .toString();
        System.out.println(data);
    }
}
```

메소드 체이닝 패턴

ABCEF

✔ StringTokenizer 클래스

- 문자열에 여러 종류가 아닌 한 종류의 구분자만 있다면 StringTokenizer를 사용할 수도 있음
StringTokenizer 객체를 생성 시 첫 번째 매개값으로 전체 문자열을 주고, 두 번째 매개값으로 구분자를 줌.
구분자를 생략하면 공백이 기본 구분자가 됨

```
String data = "홍길동/이수홍/박연수";  
StringTokenizer st = new StringTokenizer(data, "/");
```

리턴 타입	메소드(매개변수)	설명
int	countTokens()	분리할 수 있는 문자열의 총 수
boolean	hasMoreTokens()	남아 있는 문자열이 있는지 여부
String	nextToken()	문자열을 하나씩 가져옴

StringTokenizerExample.java

```
package ch12.sec05;

import java.util.StringTokenizer;

public class StringTokenizerExample {
    public static void main(String[] args) {
        String data1 = "홍길동&이수홍,박연수";
        String[] arr = data1.split("&|,");

        for(String token : arr) {
            System.out.println(token);
        }
        System.out.println();

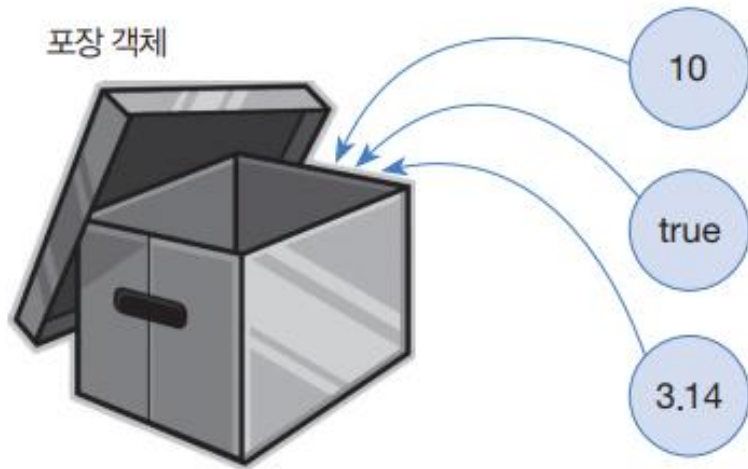
        String data2 = "홍길동/이수홍/박연수";
        StringTokenizer st = new StringTokenizer(data2, "/");
        while (st.hasMoreTokens()) {
            String token = st.nextToken();
            System.out.println(token);
        }
    }
}
```

홍길동
이수홍
박연수

홍길동
이수홍
박연수

✓ 포장 객체(Wrapper class, 래퍼클래스)

- 기본 타입(byte, char, short, int, long, float, double, boolean)의 값을 갖는 객체
- 기본 타입과 관련된 처리를 하는 메서드(기능)를 가지도록 클래스를 만듦.
- Integer.parseInt(), Float.parseFloat() 등
- 포장하고 있는 기본 타입의 값을 변경할 수 없고, 단지 객체로 생성하는 목적



기본 타입	포장 클래스
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

✓ 박싱과 언박싱

- 박싱: 기본 타입 값을 포장 객체로 만드는 과정. 포장 클래스 변수에 기본 타입 값이 대입 시 발생
- 언박싱: 포장 객체에서 기본 타입 값을 얻어내는 과정. 기본 타입 변수에 포장 객체가 대입 시 발생

```
Integer obj = 100;    //박싱  
int value = obj;      //언박싱
```

```
int value = obj + 50;  //언박싱 후 연산
```

```
ArrayList list = new ArrayList();  
list.add(100);  
//Object ← Integer ← int  
//Object ← Integer (자동 형변환)  
//Integer ← int (오토 박싱)
```

BoxingUnBoxingExample.java

```
package ch12.sec06;

public class BoxingUnBoxingExample {
    public static void main(String[] args) {
        //Boxing
        Integer obj = 100;
        System.out.println("value: " + obj.intValue());

        //Unboxing
        int value = obj;
        System.out.println("value: " + value);

        //연산 시 Unboxing
        int result = obj + 100;
        System.out.println("result: " + result);
    }
}
```

```
value: 100
value: 100
result: 200
```

✓ 문자열을 기본 타입 값으로 변환

- 포장 클래스는 문자열을 기본 타입 값으로 변환할 때도 사용.
- 대부분의 포장 클래스에는 'parse+기본타입' 명으로 되어 있는 정적 메소드 있음

✓ Math 클래스

○ 수학 계산에 사용할 수 있는 정적 메소드 제공

구분	코드	리턴값
절대값	int v1 = Math.abs(-5); double v2 = Math.abs(-3.14);	v1 = 5 v2 = 3.14
올림값	double v3 = Math.ceil(5.3); double v4 = Math.ceil(-5.3);	v3 = 6.0 v4 = -5.0
버림값	double v5 = Math.floor(5.3); double v6 = Math.floor(-5.3);	v5 = 5.0 v6 = -6.0
최대값	int v7 = Math.max(5, 9); double v8 = Math.max(5.3, 2.5);	v7 = 9 v8 = 5.3
최소값	int v9 = Math.min(5, 9); double v10 = Math.min(5.3, 2.5);	v9 = 5 v10 = 2.5
랜덤값	double v11 = Math.random();	$0.0 \leq v11 < 1.0$
반올림값	long v14 = Math.round(5.3); long v15 = Math.round(5.7);	v14 = 5 v15 = 6

MathExample.java

```
package ch12.sec07;

public class MathExample {
    public static void main(String[] args) {
        //큰 정수 또는 작은 정수 얻기
        double v1 = Math.ceil(5.3);
        double v2 = Math.floor(5.3);
        System.out.println("v1=" + v1);
        System.out.println("v2=" + v2);

        //큰값 또는 작은값 얻기
        long v3 = Math.max(3, 7);
        long v4 = Math.min(3, 7);
        System.out.println("v3=" + v3);
        System.out.println("v4=" + v4);

        //소수 이하 두 자리 얻기
        double value = 12.3456;
        double temp1 = value * 100;
        long temp2 = Math.round(temp1);
        double v5 = temp2 / 100.0;
        System.out.println("v5=" + v5);
    }
}
```

```
v1=6.0
v2=5.0
v3=7
v4=3
v5=12.35
```


✓ 랜덤 숫자 얻기

- **Math.random()**
 - 0.0과 1.0 사이의 double 타입 난수를 리턴
- **생성자**

객체 생성	설명
Random()	현재 시간을 이용해서 종자값을 자동 설정한다.
Random(long seed)	주어진 종자값을 사용한다.

○ 메소드

리턴값	메소드(매개변수)	설명
boolean	nextBoolean()	boolean 타입의 난수를 리턴
double	nextDouble()	double 타입의 난수를 리턴($0.0 \leq \sim < 1.0$)
int	nextInt()	int 타입의 난수를 리턴($-2^{32} \leq \sim < 2^{32}-1$);
int	nextInt(int n)	int 타입의 난수를 리턴($0 \leq \sim < n$)

RandomExample.java

```
package ch12.sec07;

import java.util.Arrays;
import java.util.Random;

public class RandomExample {
    public static void main(String[] args) {
        //선택번호
        int[] selectNumber = new int[6];
        Random random = new Random(3);
        System.out.print("선택번호: ");
        for(int i=0; i<6; i++) {
            selectNumber[i] = random.nextInt(45) + 1;
            System.out.print(selectNumber[i] + " ");
        }
        System.out.println();
    }
}
```

선택번호: 15 21 16 17 34 28

RandomExample.java

```
//당첨번호
int[] winningNumber = new int[6];
random = new Random(5);
System.out.print("당첨번호: ");
for(int i=0; i<6; i++) {
    winningNumber[i] = random.nextInt(45) + 1;
    System.out.print(winningNumber[i] + " ");
}
System.out.println();

//당첨여부, 비교하기 전에 배열 항목을 정렬
Arrays.sort(selectNumber);
Arrays.sort(winningNumber);
boolean result = Arrays.equals(selectNumber, winningNumber); // 배열 항목 비교
System.out.print("당첨여부: ");
if(result) {
    System.out.println("1등에 당첨되었습니다.");
} else {
    System.out.println("당첨되지 않았습니다.");
}
}
```

당첨번호: 18 38 45 15 22 36
당첨여부: 당첨되지 않았습니다.

8 날짜와 시간 클래스

✓ java.util 패키지

클래스	설명
Date	날짜 정보를 전달하기 위해 사용
Calendar	다양한 시간대별로 날짜와 시간을 얻을 때 사용
LocalDateTime	날짜와 시간을 조작할 때 사용

✓ Date 클래스

- 날짜를 표현하는 클래스로 객체 간에 날짜 정보를 주고받을 때 사용
- Date() 생성자는 컴퓨터의 현재 날짜를 읽어 Date 객체로 만듦

```
Date now = new Date();
```

 DateExample.java

```
package ch12.sec08;

import java.text.*;
import java.util.*;

public class DateExample {
    public static void main(String[] args) {
        Date now = new Date();
        String strNow1 = now.toString();
        System.out.println(strNow1);

        SimpleDateFormat sdf = new SimpleDateFormat("yyyy.MM.dd HH:mm:ss");
        String strNow2 = sdf.format(now);
        System.out.println(strNow2);
    }
}
```

```
Thu Jan 18 12:22:20 KST 2024
2024.01.18 12:22:20
```

8 날짜와 시간 클래스

✓ Calendar 클래스

- 달력을 표현하는 추상 클래스
- getInstance() 메소드로 컴퓨터에 설정된 시간대 기준으로 Calendar 하위 객체를 얻을 수 있음

```
Calendar now = Calendar.getInstance();
```

```
int year    = now.get(Calendar.YEAR);           //년도를 리턴
int month   = now.get(Calendar.MONTH) + 1;      //월을 리턴
int day     = now.get(Calendar.DAY_OF_MONTH);   //일을 리턴
int week    = now.get(Calendar.DAY_OF_WEEK);    //요일을 리턴
int amPm    = now.get(Calendar.AM_PM);         //오전/오후를 리턴
int hour    = now.get(Calendar.HOUR);          //시를 리턴
int minute  = now.get(Calendar.MINUTE);        //분을 리턴
int second  = now.get(Calendar.SECOND);        //초를 리턴
```

CalendarExample.java

```
package ch12.sec08;

import java.util.*;

public class CalendarExample {
    public static void main(String[] args) {
        Calendar now = Calendar.getInstance();

        int year  = now.get(Calendar.YEAR);
        int month = now.get(Calendar.MONTH) + 1;
        int day   = now.get(Calendar.DAY_OF_MONTH);

        int week  = now.get(Calendar.DAY_OF_WEEK);
        String strWeek = null;
        switch(week) {
            case Calendar.MONDAY:    strWeek = "월"; break;
            case Calendar.TUESDAY:  strWeek = "화"; break;
            case Calendar.WEDNESDAY: strWeek = "수"; break;
            case Calendar.THURSDAY: strWeek = "목"; break;
            case Calendar.FRIDAY:   strWeek = "금"; break;
            case Calendar.SATURDAY: strWeek = "토"; break;
            default:                 strWeek = "일";
        }
    }
}
```


 CalendarExample.java

```
int amPm = now.get(Calendar.AM_PM);
String strAmPm = null;
if(amPm == Calendar.AM) {
    strAmPm = "오전";
} else {
    strAmPm = "오후";
}
```

```
int hour = now.get(Calendar.HOUR);
int minute = now.get(Calendar.MINUTE);
int second = now.get(Calendar.SECOND);
```

```
System.out.print(year + "년 ");
System.out.print(month + "월 ");
System.out.println(day + "일 ");
System.out.print(strWeek + "요일 ");
System.out.println(strAmPm + " ");
System.out.print(hour + "시 ");
System.out.print(minute + "분 ");
System.out.println(second + "초 ");
```

```
}
}
```

2024년 1월 18일
목요일 오후
0시 25분 0초

8 날짜와 시간 클래스

✓ 날짜와 시간 조작

- java.time 패키지의 LocalDateTime 클래스가 제공하는 메소드를 이용해 날짜와 시간을 조작 가능
- 책 대여일이 20일 경우 plusDays()로 해당 날짜를 구할 수 있음.
- 날짜/시각을 원하는 포맷으로 출력 가능

```
LocalDateTime now = LocalDateTime.now();
```

메소드(매개변수)	설명
minusYears(long)	년 빼기
minusMonths(long)	월 빼기
minusDays(long)	일 빼기
minusWeeks(long)	주 빼기
plusYears(long)	년 더하기
plusMonths(long)	월 더하기
plusWeeks(long)	주 더하기
plusDays(long)	일 더하기
minusHours(long)	시간 빼기
minusMinutes(long)	분 빼기
minusSeconds(long)	초 빼기
minusNanos(long)	나노초 빼기
plusHours(long)	시간 더하기
plusMinutes(long)	분 더하기
plusSeconds(long)	초 더하기

DateTimeOperationExample.java

```
package ch12.sec08;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class DateTimeOperationExample {
    public static void main(String[] args) {
        LocalDateTime now = LocalDateTime.now(); //년월일 시분초 구해온다.
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern(" yyyy.MM.dd a HH:mm:ss "); //패턴 설정하는 객체
        System.out.println( " 현재 시간: " + now.format(dtf)); //패턴 설정하는 객체를 년월일 시분초에 적용

        LocalDateTime result1 = now.plusYears(1);
        System.out.println("1년 덧셈: " + result1.format(dtf));

        LocalDateTime result2 = now.minusMonths(2);
        System.out.println("2월 뺄셈: " + result2.format(dtf));

        LocalDateTime result3 = now.plusDays(7);
        System.out.println("7일 덧셈: " + result3.format(dtf));
    }
}
```

다양하게 조작 가능

현재 시간: 2024.01.18 오후 12:28:40
1년 덧셈: 2025.01.18 오후 12:28:40
2월 뺄셈: 2023.11.18 오후 12:28:40
7일 덧셈: 2024.01.25 오후 12:28:40

8 날짜와 시간 클래스

✓ 날짜와 시간 비교

- LocalDateTime 클래스는 날짜와 시간을 비교할 수 있는 메소드 제공

리턴 타입	메소드(매개변수)	설명
boolean	isAfter(other)	이후 날짜인지?
	isBefore(other)	이전 날짜인지?
	isEqual(other)	동일 날짜인지?
long	until(other, unit)	주어진 단위(unit) 차이를 리턴

- 특정 날짜, 시간 만들기

```
LocalDateTime target = LocalDateTime.of(year, month, dayOfMonth, hour, minute,
second);
```

DateTimeCompareExample.java

```
package ch12.sec08;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class DateTimeCompareExample {
    public static void main(String[] args) {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy.MM.dd a HH:mm:ss");

        LocalDateTime startDateTime = LocalDateTime.of(2021, 1, 1, 0, 0, 0);
        System.out.println("시작일: " + startDateTime.format(dtf));

        LocalDateTime endDateTime = LocalDateTime.of(2021, 12, 31, 0, 0, 0);
        System.out.println("종료일: " + endDateTime.format(dtf));

        if(startDateTime.isBefore(endDateTime)) {
            System.out.println("진행 중입니다.");
        } else if(startDateTime.isEqual(endDateTime)) {
            System.out.println("종료합니다.");
        } else if(startDateTime.isAfter(endDateTime)) {
            System.out.println("종료했습니다.");
        }
    }
}
```

시작일: 2021.01.01 오전 00:00:00
종료일: 2021.12.31 오전 00:00:00
진행 중입니다.

DateTimeCompareExample.java

```
long remainYear = startDateTime.until(endDateTime, ChronoUnit.YEARS);
long remainMonth = startDateTime.until(endDateTime, ChronoUnit.MONTHS);
long remainDay = startDateTime.until(endDateTime, ChronoUnit.DAYS);
long remainHour = startDateTime.until(endDateTime, ChronoUnit.HOURS);
long remainMinute = startDateTime.until(endDateTime, ChronoUnit.MINUTES);
long remainSecond = startDateTime.until(endDateTime, ChronoUnit.SECONDS);

System.out.println("남은 해: " + remainYear);
System.out.println("남은 월: " + remainMonth);
System.out.println("남은 일: " + remainDay);
System.out.println("남은 시간: " + remainHour);
System.out.println("남은 분: " + remainMinute);
System.out.println("남은 초: " + remainSecond);
}
```

```
남은 해: 0
남은 월: 11
남은 일: 364
남은 시간: 8736
남은 분: 524160
남은 초: 31449600
```

✓ DecimalFormat

- 숫자를 형식화된 문자열로 변환 → 원하는 포맷으로 출력 가능

기호	의미	패턴 예	1234567.89 → 변환 결과
0	10진수(빈자리는 0으로 채움)	0 0.0 0000000000.00000	1234568 1234567.9 0001234567.89000
#	10진수(빈자리는 채우지 않음)	# #. # #####.#####	1234568 1234567.9 1234567.89
.	소수점	#.0	1234567.9
-	음수 기호	+#.0 -#.0	+1234567.9 -1234567.9
,	단위 구분	#,###.0	1,234,567.9
E	지수 문자	0.0E0	1.2E6
;	양수와 음수의 패턴을 모두 기술할 경우, 패턴 구분자	+#,### ; -#,###	+1,234,568(양수일 때) -1,234,568(음수일 때)
%	% 문자	## %	123456789 %
\u00A4	통화 기호	\u00A4 #,###	₩1,234,568

✓ Format 형식 클래스

- 숫자 또는 날짜를 원하는 형태의 문자열로 변환해주는 기능 제공
- `java.text` 패키지에 정의

Format 클래스	설명
<code>DecimalFormat</code>	숫자를 형식화된 문자열로 변환
<code>SimpleDateFormat</code>	날짜를 형식화된 문자열로 변환

✓ DecimalFormat

- 숫자를 형식화된 문자열로 변환하는 기능 제공

```
DecimalFormat df = new DecimalFormat("#,###.0");  
String result = df.format(1234567.89); //1,234,567.9
```

✓ DecimalFormat

○ 변환 패턴

기호	의미	패턴 예	1234567.89 → 변환 결과
0	10진수(빈자리는 0으로 채움)	0 0.0 0000000000.00000	1234568 1234567.9 0001234567.89000
#	10진수(빈자리는 채우지 않음)	# #. # #####. #####	1234568 1234567.9 1234567.89
.	소수점	#.0	1234567.9
-	음수 기호	+#.0 -#.0	+1234567.9 -1234567.9
,	단위 구분	#,###.0	1,234,567.9
E	지수 문자	0.0E0	1.2E6
;	양수와 음수의 패턴을 모두 기술할 경우, 패턴 구분자	+#,### ; -#,###	+1,234,568(양수일 때) -1,234,568(음수일 때)
%	% 문자	#,# %	123456789 %
\u00A4	통화 기호	\u00A4 #,###	₩1,234,568

DecimalFormatExample.java

```
package ch12.sec09;

import java.text.DecimalFormat;

public class DecimalFormatExample {
    public static void main(String[] args) {
        double num = 1234567.89;

        DecimalFormat df;

        //정수 자리까지 표기
        df = new DecimalFormat("#,###");
        System.out.println( df.format(num) );

        //무조건 소수 첫째 자리까지 표기
        df = new DecimalFormat("#,###.0");
        System.out.println( df.format(num) );
    }
}
```

```
1,234,568
1,234,567.9
```

✓ SimpleDateFormat

○ 날짜를 형식화된 문자열로 변환

패턴 문자	의미	패턴 문자	의미
y	년	H	시(0~23)
M	월	h	시(1~12)
d	일	K	시(0~11)
D	월 구분이 없는 일(1~365)	k	시(1~24)
E	요일	m	분
a	오전/오후	s	초
w	년의 몇 번째 주	S	밀리세컨드(1/1000초)
W	월의 몇 번째 주		

```
SimpleDateFormat sdf = new SimpleDateFormat("yyyy년 MM월 dd일");
String strDate = sdf.format(new Date()); //2021년 11월 28일
```

SimpleDateFormatExample.java

```
package ch12.sec09;

import java.text.SimpleDateFormat;
import java.util.Date;

public class SimpleDateFormatExample {
    public static void main(String[] args) {
        Date now = new Date();

        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        System.out.println( sdf.format(now) );

        sdf = new SimpleDateFormat("yyyy년 MM월 dd일");
        System.out.println( sdf.format(now) );

        sdf = new SimpleDateFormat("yyyy.MM.dd a HH:mm:ss");
        System.out.println( sdf.format(now) );

        sdf = new SimpleDateFormat("오늘은 E요일");
        System.out.println( sdf.format(now) );
    }
}
```

```
2024-01-18
2024년 01월 18일
2024.01.18 오후 12:38:11
오늘은 목요일
```

SimpleDateFormatExample.java

```
sdf = new SimpleDateFormat("올해의 D번째 날");  
System.out.println( sdf.format(now) );  
  
sdf = new SimpleDateFormat("이달의 d번째 날");  
System.out.println( sdf.format(now) );  
}  
}
```

오늘은 목요일
올해의 18번째 날
이달의 18번째 날

✓ 정규 표현식

○ 문자 또는 숫자와 관련된 표현과 반복 기호가 결합된 문자열

표현 및 기호	설명		
[]	한 개의 문자	[abc]	a, b, c 중 하나의 문자
		[^abc]	a, b, c 이외의 하나의 문자
		[a-zA-Z]	a~z, A~Z 중 하나의 문자
\d	한 개의 숫자, [0-9]와 동일		
\s	공백		
\w	한 개의 알파벳 또는 한 개의 숫자, [a-zA-Z_0-9]와 동일		
\.	.		
.	모든 문자 중 한 개의 문자		
?	없음 또는 한 개		
*	없음 또는 한 개 이상		
+	한 개 이상		
{n}	정확히 n개		
{n,}	최소한 n개		
{n, m}	n개부터 m개까지		
a b	a 또는 b		
()	그룹핑		

✓ 정규 표현식

○ 전화번호를 위한 정규 표현식

- 02-123-1234
- 02-124-5648

`(02|010)-\d{3,4}-\d{4}`

02나 010
-문자
숫자 3자리에서 4자리
-문자
숫자 4자리

○ 이메일을 위한 정규 표현식

- white@naver.com

`\w+@(\w+\. \w+)`

문자를 하나 이상
@문자
문자를 하나 이상
.문자
문자를 하나 이상
문자 문자를 하나 이상이 0 또는 1

10 정규 표현식 클래스

✓ Pattern 클래스로 검증

- java.util.regex 패키지의 Pattern 클래스
 - 정규 표현식으로 문자열을 검증하는 matches() 메소드 제공

```
boolean result = Pattern.matches("정규식", "검증할 문자열");
```

 PatternExample.java

```
package ch12.sec10;
import java.util.regex.Pattern;

public class PatternExample {
    public static void main(String[] args) {
        String regExp = "(02|010)-\\W\\Wd{3,4}-\\W\\Wd{4}";
        String data = "010-123-4567";
        boolean result = Pattern.matches(regExp, data);
        if(result) {
            System.out.println("정규식과 일치합니다.");
        } else {
            System.out.println("정규식과 일치하지 않습니다.");
        }

        regExp = "\\W\\Ww+@\\W\\Ww+\\W\\W.\\W\\Ww+(\\W\\W.\\W\\Ww+)?";
        data = "angel@mycompanycom";
        result = Pattern.matches(regExp, data);
        if(result) {
            System.out.println("정규식과 일치합니다.");
        } else {
            System.out.println("정규식과 일치하지 않습니다.");
        }
    }
}
```

정규식과 일치합니다.
정규식과 일치하지 않습니다.